
Explotación de una Base de Datos utilizando Jakarta Persistence



ASIGNATURA: BASES DE DATOS 2

PRÁCTICA 4

CURSO: 2025/26

COORDINADORA: Vázquez Martín, Lucía (NIP 871886)

Zhou Chen, Luna (NIP 812103)

Índice

Índice.....	2
1. Introducción.....	4
2. Entorno de trabajo y organización de los proyectos.....	4
2.1. Creación y organización de los proyectos Maven.....	4
2.2. Gestión de dependencias y configuración base.....	5
3. Generación del esquema relacional a partir del esquema de clases Java.....	6
3.1. Mapeo objeto-relacional.....	6
3.2. Configuración del entorno de persistencia.....	8
3.3. Población de datos y simulación de operaciones.....	8
3.4. Ejecución de las pruebas.....	9
3.5. Análisis del esquema relacional generado.....	9
3.6. Análisis comparativo de consultas: JPQL frente a SQL Nativo.....	10
4. Implementación del mapeo objeto/relacional sobre esquema relacional preexistente.....	11
4.1. Objetivo y planteamiento de la parte 2.....	11
4.2. Preparación del entorno Oracle, conexión con el esquema relacional previo y configuración de persistencia.....	11
4.3. Definición de las entidades Java.....	14
4.4. Mapeo de relaciones y clave compuesta.....	15
4.5. Clase de prueba Test.java: inserción, validación y consultas.....	16
4.6. Consultas de ejemplo.....	18
4.6.1. Consulta C1: saldo total acumulado de cada cliente.....	18
4.6.2. Consulta C2: historial de transferencias superiores a un importe dado.....	18
4.6.3. Consulta C3: saldo medio de las cuentas corrientes por oficina.....	19
5. Conclusiones.....	20
6. Dificultades y lecciones aprendidas.....	21
6.1. Correspondencia nomenclatura entre modelo de objetos y esquema relacional.....	21
6.2. Representación de claves primarias compuestas en entorno orientado a objetos.....	22
7. Organización del trabajo y esfuerzos invertidos.....	23
8. Bibliografía.....	24
9. Anexos.....	25
9.1. Anexo I. Código fuente y ficheros de configuración.....	25
9.1.1. Proyecto java-gen-esquema.....	25
java-gen-esquema/pom.xml.....	25
java-gen-esquema/.../persistence.xml.....	25
java-gen-esquema/.../Cliente.java.....	26
java-gen-esquema/.../Cuenta.java.....	27
java-gen-esquema/.../CuentaAhorro.java.....	29
java-gen-esquema/.../CuentaCorriente.java.....	29
java-gen-esquema/.../Oficina.java.....	29
java-gen-esquema/.../Operacion.java.....	30
java-gen-esquema/.../OperacionEfectivo.java.....	31

java-gen-esquema/.../OperacionId.java.....	32
java-gen-esquema/.../OperacionTransferencia.java.....	33
java-gen-esquema/.../Test.java.....	33
9.1.2. Proyecto java-previo-esquema.....	36
java-previo-esquema/pom.xml.....	36
java-previo-esquema/.../persistence.xml.....	37
java-previo-esquema/.../Cliente.java.....	37
java-previo-esquema/.../Cuenta.java.....	39
java-previo-esquema/.../Oficina.java.....	41
java-previo-esquema/.../Operacion.java.....	42
java-previo-esquema/.../OperacionId.java.....	44
java-previo-esquema/.../Test.java.....	45
9.2. Anexo II. Salidas de ejecución y comprobaciones.....	50
9.2.1. Salida de creación del esquema relacional Banquito en Oracle.....	50
9.2.2. Salida de pruebas del esquema relacional Banquito.....	51
9.2.3. Verificación de tablas disponibles en Oracle.....	53
9.2.4. Salida de ejecución de Test.java en java-gen-esquema.....	53
9.2.5. Salida de ejecución de Test.java en java-previo-esquema.....	55

1. Introducción

En la presente **Práctica 4** se aborda el uso de **Jakarta Persistence** (JPA) para realizar el **mapeo objeto-relacional** entre un modelo de clases en Java y un esquema de base de datos relacional. Para ello, se emplea **Hibernate** como herramienta de persistencia, permitiendo automatizar parte del proceso de traducción entre ambos paradigmas.

El trabajo se centra, por un lado, en la **generación automática** de un esquema relacional a partir de clases Java y, por otro, en el **establecimiento de correspondencias** entre dichas clases y un esquema relacional previamente definido en prácticas anteriores. De este modo, se analizan tanto el proceso de generación como la adaptación a un diseño existente.

Finalmente, se realizan **consultas sobre los datos persistidos** utilizando distintos mecanismos, lo que permite evaluar el funcionamiento del mapeo y comparar diferentes formas de acceso a la información.

2. Entorno de trabajo y organización de los proyectos

En la Práctica 2, el desarrollo de **programas en Java** se había abordado mediante un enfoque sencillo basado en la edición manual de ficheros y su compilación directa desde terminal utilizando **comandos** como **javac**. Este procedimiento resultaba suficiente en contextos reducidos, donde el número de dependencias externas era muy limitado. Sin embargo, en la presente práctica, centrada en Jakarta Persistence y Hibernate, este enfoque deja de ser adecuado, ya que la aplicación requiere integrar varias librerías externas y mantener una configuración consistente del classpath. Por este motivo, se adoptó un entorno de desarrollo basado en **Visual Studio Code (VSC) junto con Maven**, lo que permitió automatizar la resolución de dependencias, estructurar correctamente los proyectos y simplificar la compilación y ejecución de las pruebas.

Con el fin de ajustarse a la organización exigida por el enunciado, se decidió separar el trabajo en **dos proyectos independientes**: uno para la **generación automática** del esquema relacional a partir de clases Java y otro para el **mapeo** sobre un esquema relacional preexistente. Esta separación facilita la reproducibilidad, evita mezclar configuraciones incompatibles y permite preparar con claridad los paquetes requeridos en la entrega, **java-gen-esquema.zip** y **java-previo-esquema.zip**.

2.1. Creación y organización de los proyectos Maven

La **creación de los proyectos** se realizó directamente desde VSC, utilizando el asistente integrado para proyectos Java. El procedimiento seguido fue el siguiente:

1. Se instaló en VSC la extensión **Extension Pack for Java**, que proporciona soporte para proyectos Java, ejecución, depuración y resolución de dependencias Maven.
2. Se abrió el **buscador de comandos** con la combinación **Ctrl + Shift + P**.
3. Se seleccionó la opción **Java: Create Java Project**.
4. En el asistente, se eligió **Maven** como sistema de gestión del proyecto.

5. A continuación, se seleccionó la opción **No Archetype**, con el objetivo de partir de una estructura mínima y controlada.
6. Como **identificador de grupo** se utilizó el valor: **es.unizar.bd2**
7. Para el **identificador de artefacto** se crearon dos proyectos distintos, cada uno asociado a una de las dos partes principales de la práctica:
 - **java-gen-esquema**
 - **java-previo-esquema**
8. Finalmente, se seleccionó el **directorio de trabajo** y se abrió cada proyecto en el entorno.

Una vez finalizado este proceso, VSC generó automáticamente la estructura estándar de Maven, incluyendo los directorios **src/main/java** para el código fuente y **src/main/resources** para los ficheros de configuración.

Con el objetivo de mantener una organización homogénea, en ambos proyectos se utilizó el paquete base:

```
package es.unizar.bd2
```

De este modo, las clases Java quedaron ubicadas dentro de la ruta:

```
src/main/java/es/unizar/bd2/
```

mientras que los ficheros de configuración de persistencia se colocaron en:

```
src/main/resources/META-INF/
```

2.2. Gestión de dependencias y configuración base

Tras generar cada proyecto, fue necesario completar el fichero **pom.xml** con las dependencias necesarias para trabajar con Jakarta Persistence, Hibernate y Oracle JDBC. Esta configuración constituye la base común de ambas partes de la práctica, ya que los dos proyectos requieren **una implementación de JPA, un proveedor ORM y un controlador JDBC** adecuado para Oracle.

Para ello, se añadió el **bloque <dependencies>** dentro del **elemento <project>**, después de la **sección <properties>**. La configuración utilizada se puede encontrar en [java-gen-esquema/pom.xml](#) y en [java-previo-esquema/pom.xml](#).

Una vez añadidas las dependencias, resultó necesario **comprobar** que Maven había descargado e integrado correctamente las **librerías en el entorno**. Para ello, se realizaron dos verificaciones complementarias.

En primer lugar, se llevó a cabo una **comprobación visual** desde el propio entorno de VSC. El procedimiento fue el siguiente:

1. Se accedió al **panel lateral** del explorador.
2. En la sección **Java Projects**, se desplegó el proyecto creado.
3. Dentro del apartado **Maven Dependencies**, se verificó la presencia de las librerías correspondientes a Jakarta Persistence, Hibernate y Oracle JDBC.

La aparición de estas dependencias confirmó que el proyecto había sido **reconocido correctamente** como proyecto Maven y que las librerías necesarias estaban disponibles para el compilador.

En segundo lugar, se realizó una **verificación práctica** mediante una **clase temporal de prueba**. Para ello:

1. Se creó un **fichero Java** de prueba dentro de **src/main/java/es/unizar/bd2/**.
2. En dicho fichero se importaron **clases representativas** de las dependencias instaladas, por ejemplo:

```
package es.unizar.bd2;

import org.hibernate.Session;
import oracle.jdbc.OracleDriver;

public class Prueba {
}
```

Si las dependencias no hubieran sido resueltas correctamente, el editor habría marcado los imports como erróneos. Sin embargo, **al no aparecer errores**, se concluyó que las librerías estaban correctamente enlazadas y listas para su uso.

3. Generación del esquema relacional a partir del esquema de clases Java

3.1. Mapeo objeto-relacional

Como punto de partida para el desarrollo del mapeo objeto-relacional se ha tomado el **modelo conceptual de datos** definido en la Práctica 2, el cual ha sido traducido a un conjunto de clases Java siguiendo el estándar de **Jakarta Persistence** (JPA). Este enfoque permite establecer una correspondencia directa entre las entidades del dominio y su representación en el código, delegando en el framework Hibernate la generación y gestión del esquema relacional subyacente.

De este modo, las **entidades principales** del sistema, como *Cliente*, *Oficina*, *Cuenta* y *Operacion*, se han modelado como clases Java anotadas con **@Entity**. Sus **atributos** se corresponden directamente con los definidos en el modelo conceptual, utilizándose la anotación **@Id** para indicar las **claves primarias**.

Un aspecto relevante del mapeo es el tratamiento de la entidad *Operacion*, cuya **clave primaria es compuesta**. Dicha clave está formada por el código de la operación y la referencia a la cuenta origen, lo que refleja la dependencia de identificación establecida en el modelo conceptual. Para su implementación se ha utilizado la anotación **@IdClass**, junto con la **clase auxiliar OperacionId**, que encapsula los atributos que conforman la clave. Este mecanismo permite a JPA gestionar correctamente las claves compuestas y garantiza que, a nivel relacional, se genere una restricción de clave primaria sobre ambos atributos.

En cuanto a la representación de las **jerarquías de especialización**, tanto en el caso de *Cuenta* (con los subtipos *CuentaAhorro* y *CuentaCorriente*) como en *Operacion* (con *OperacionTransferencia* y *OperacionEfectivo*), se ha optado por la estrategia de **tabla única (InheritanceType.SINGLE_TABLE)**. Esta decisión es coherente con la adoptada en el diseño relacional de la Práctica 2, donde ya se utilizaba una única relación para cada jerarquía. Mediante la anotación **@DiscriminatorColumn**, se introduce un atributo discriminador que permite diferenciar los distintos subtipos dentro de la misma tabla, mientras que las anotaciones **@DiscriminatorValue** permiten asociar cada clase concreta con su valor correspondiente.

Por otro lado, las **asociaciones entre entidades** se han implementado utilizando las anotaciones proporcionadas por JPA, respetando las **cardinalidades** definidas en el modelo conceptual. Las relaciones de tipo **uno a muchos** se han modelado mediante **@ManyToOne** y **@OneToMany**, como ocurre, por ejemplo, entre *CuentaCorriente* y *Oficina*, o entre *Operacion* y *Cuenta* (cuenta origen).

Cabe destacar la **relación de titularidad** entre *Cliente* y *Cuenta*, que en el modelo conceptual se definía como una relación de **muchos a muchos**. Esta se ha implementado mediante la notación **@ManyToMany**, apoyada por **@JoinTable** para especificar explícitamente la **tabla intermedia (ser_titular)** y sus claves ajenas. De este modo, se mantiene la semántica original del modelo sin necesidad de introducir clases adicionales, delegando en el framework ORM la gestión de la tabla de asociación.

Asimismo, se ha definido la **relación bidireccional** entre *Cliente* y *Cuenta*, utilizando el atributo **mappedBy** en la clase *Cuenta* para indicar que la gestión de la relación recae sobre la entidad *Cliente*. Esto permite una navegación eficiente en ambos sentidos dentro del modelo orientado a objetos.

En conjunto, el **mapeo realizado** permite trasladar de forma fiel el modelo conceptual al entorno de programación, manteniendo tanto la estructura como las restricciones principales del dominio. Además, el **uso de Jakarta Persistence** facilita la **abstracción** del modelo relacional, permitiendo que la interacción con la base de datos se realice a través de objetos, sin necesidad de trabajar directamente con sentencias SQL.

El **código fuente completo** correspondiente a las clases Java desarrolladas para este mapeo se adjunta en [Anexo I](#).

3.2. Configuración del entorno de persistencia

La comunicación entre la aplicación Java y el sistema gestor de bases de datos (SGBD) Oracle (alojado en el servidor de la universidad) se ha centralizado mediante el fichero de configuración [persistence.xml](#). Para la elaboración de este documento se ha tomado como referencia el fichero proporcionado por el profesorado (**persistence.xml** incluido en el proyecto comprimido **simple-hibernate-7.3-test.zip**), a partir del cual se ha definido una unidad de persistencia adaptada a las necesidades de la aplicación.

En dicha unidad de persistencia se ha especificado el **proveedor de persistencia** (Hibernate), así como el **dialecto correspondiente a Oracle**, lo que permite al framework generar sentencias SQL compatibles con el SGBD utilizado. Asimismo, se han configurado los **parámetros de conexión JDBC**, incluyendo la URL de acceso, el usuario y la contraseña, necesarios para establecer la conexión con la base de datos remota.

Un aspecto especialmente relevante de la configuración es el uso de la propiedad **hibernate.hbm2ddl.auto** con el valor **create**. Esta configuración indica al motor ORM que, al iniciar la aplicación, elimine el esquema existente y genere automáticamente uno nuevo a partir de las clases anotadas. En la práctica, esto implica la **ejecución de sentencias de borrado** (DROP) y **creación** (CREATE) de las tablas correspondientes.

La utilización de esta estrategia permite **garantizar la independencia** respecto al estado previo de la base de datos, evitando conflictos derivados de la existencia de tablas o restricciones anteriores. Además, **simplifica el proceso de desarrollo y pruebas**, al automatizar completamente la creación del esquema, eliminando la necesidad de mantener scripts manuales de inicialización.

3.3. Población de datos y simulación de operaciones

Para la **verificación del correcto funcionamiento** del mapeo diseñado, se ha desarrollado una clase principal ([Test.java](#)), encargada de instanciar las entidades y gestionar su persistencia mediante el uso del **EntityManager**. El proceso de prueba se ha estructurado en distintas transacciones, con el objetivo de garantizar la consistencia de los datos almacenados en la base de datos.

En una **primera transacción** se ha llevado a cabo la **población de datos iniciales**, creando instancias de *Oficina*, *Cliente* y los distintos tipos de *Cuenta*. La **inserción de registros en la tabla intermedia ser_titular** se realiza de forma transparente desde el punto de vista del código, es decir, basta con añadir las cuentas a la colección correspondiente del cliente antes de invocar el método **persist()**, delegando en el framework ORM la **generación automática de las sentencias INSERT** necesarias en la tabla de asociación.

En una **segunda transacción** se han **simulado operaciones bancarias** habituales, tales como retiradas de efectivo y transferencias entre cuentas. Esta fase pone de manifiesto cómo el modelo orientado a objetos permite **expresar la lógica de negocio directamente** sobre los objetos en memoria (por ejemplo, la actualización del saldo de las cuentas implicadas), siendo **posteriormente sincronizada** con la base de datos a través

del contexto de persistencia. De este modo, el framework se encarga de **generar automáticamente las sentencias UPDATE** necesarias, garantizando además la correcta gestión de las asociaciones entre entidades, como la relación de cada operación con su cuenta de origen, cuenta de destino u oficina correspondiente.

3.4. Ejecución de las pruebas

Una vez configurado el entorno de persistencia y definida la lógica de prueba en la clase *Test.java*, se ha procedido a su **ejecución**. En el entorno de desarrollo VSC, este proceso puede **iniciarse de forma directa** mediante la **opción Run**, proporcionada por la extensión de Java sobre el método principal (*main*).

La ejecución desencadena la **compilación del proyecto y la inicialización del framework Hibernate**, que establece la conexión con la base de datos Oracle. A continuación, el motor ORM genera automáticamente el esquema relacional en tiempo de ejecución y procesa las distintas transacciones definidas en el código.

El registro detallado de la **salida obtenida por consola**, donde se documenta todo el proceso, desde el borrado y creación de tablas hasta el resultado final de las consultas comparativas, se encuentra disponible en el [Anexo II](#).

3.5. Análisis del esquema relacional generado

A partir de la ejecución descrita en el apartado anterior, Hibernate **genera automáticamente el esquema relacional** correspondiente al modelo de clases definido mediante **anotaciones de Jakarta Persistence**. Este proceso permite traducir de forma directa el modelo orientado a objetos a un esquema de base de datos sin necesidad de definir manualmente las sentencias SQL. A continuación, **se analiza el resultado obtenido** con el objetivo de comprobar su correspondencia con el diseño conceptual.

En primer lugar, se observa la **creación de las tablas principales** del sistema, que se corresponden con las entidades definidas en el modelo: *Cliente*, *Cuenta*, *Oficina* y *Operacion*. Asimismo, **se genera una tabla adicional** denominada *ser_titular*, encargada de representar la relación de muchos a muchos entre las entidades *Cliente* y *Cuenta*.

En cuanto a las **claves primarias**, cada tabla refleja correctamente el identificador definido en las clases Java. Así, la entidad *Cliente* utiliza el atributo *dni* como clave primaria, *Cuenta* utiliza el *iban*, y *Oficina* el atributo *codigo*. Por su parte, la entidad *Operacion* presenta una clave primaria compuesta formada por los atributos *codigo* y *refCuentaOrigen*, en coherencia con la definición realizada mediante **@IdClass**.

Respecto a las **relaciones entre entidades**, estas se traducen en **claves foráneas** dentro del esquema relacional. En particular, la tabla *Cuenta* incorpora una referencia a *Oficina* mediante el atributo *refOficina*, mientras que la tabla *Operacion* incluye referencias tanto a la cuenta de origen como, en su caso, a la cuenta de destino o a la oficina asociada. Por su parte, la tabla *ser_titular* contiene las claves ajenas necesarias para enlazar los clientes con sus cuentas, implementando correctamente la relación N:M definida en el modelo conceptual.

Un aspecto relevante es la **representación de las jerarquías de herencia**. La estrategia **SINGLE_TABLE** adoptada en las entidades *Cuenta* y *Operacion* se traduce en la existencia de una única tabla para cada jerarquía, diferenciando los distintos subtipos mediante columnas discriminadoras (*Es_ahorro* y *Tipo_operacion*, respectivamente). Este enfoque simplifica el esquema relacional, evitando la creación de múltiples tablas para cada subtipo.

En conjunto, el **esquema relacional generado refleja fielmente la estructura y restricciones del modelo conceptual** definido previamente. Hibernate permite automatizar este proceso de forma transparente, garantizando la coherencia entre el modelo orientado a objetos y su representación en la base de datos, y reduciendo significativamente la complejidad asociada a la definición manual del esquema.

3.6. Análisis comparativo de consultas: JPQL frente a SQL Nativo

Con el objetivo de evaluar la **capacidad de abstracción** proporcionada por Jakarta Persistence, se ha llevado a cabo un análisis comparativo entre consultas formuladas en **JPQL** (Java Persistence Query Language) y sus equivalentes en **SQL Nativo**. Analizando los resultados obtenidos, esta **comparativa** se articula en torno a tres requerimientos que ponen de manifiesto las diferencias paradigmáticas entre ambos lenguajes.

En la **primera consulta**, destinada a calcular el saldo total acumulado por cada cliente, se hace evidente la **simplificación que aporta el modelo de objetos frente al modelo físico**. Mientras que en **SQL Nativo** resulta necesario construir uniones explícitas (JOINS) con la tabla intermedia *ser_titular*, en **JPQL** es posible navegar de forma directa a través de la colección de cuentas mapeada en la entidad cliente (**JOIN c.cuentas**), ocultando por completo la existencia de la tabla de asociación.

La **segunda consulta** busca obtener un historial de operaciones de transferencia que superen un importe determinado. Debido a la estrategia de herencia **SINGLE_TABLE** adoptada, la formulación en **SQL Nativo** requiere la inclusión manual de un filtro sobre la columna discriminadora (**WHERE Tipo_operacion = 'Transferencia'**) para evitar que se recuperen operaciones de efectivo. Sin embargo, **JPQL** abstrae este detalle de implementación física; al indicar que la consulta se realiza sobre la entidad hija *OperacionTransferencia*, el motor ORM inyecta automáticamente la condición discriminadora en el código SQL generado.

Finalmente, la **tercera consulta** calcula el saldo medio de las cuentas corrientes agrupado por código de oficina. Nuevamente, **JPQL** demuestra su superioridad semántica al permitir la navegación fluida por referencias de objetos (utilizando la notación por puntos, como en **c.oficina.codigo**). Por el contrario, la versión en **SQL Nativo** exige trabajar directamente con claves ajenas y aplicar filtros manuales adicionales sobre la columna discriminadora de las cuentas para aislar las de tipo corriente (**Es_ahorro = 0**).

En conclusión, Jakarta Persistence **simplifica** notablemente la integración entre el modelo orientado a objetos y las bases de datos relacionales. Su capacidad para **automatizar** esquemas, **gestionar** claves complejas y **utilizar JPQL para abstraer** las relaciones físicas lo convierte en un estándar indispensable para desarrollar sistemas escalables y mantenibles.

4. Implementación del mapeo objeto/relacional sobre esquema relacional preexistente

4.1. Objetivo y planteamiento de la parte 2

En esta segunda parte de la práctica se aborda el **establecimiento de correspondencias** entre un conjunto de clases Java y un esquema relacional ya existente. En este caso se parte de la **base de datos relacional Banquito** desarrollada en la Práctica 2 y se construye sobre ella el mapeo objeto/relacional necesario para poder explotarla desde Java.

En lugar de diseñar libremente un modelo orientado a objetos y dejar que Hibernate lo traduzca a tablas, **se toman como referencia las tablas y relaciones ya existentes**. Por ello, las clases Java se definen de forma que reflejen la estructura relacional previa.

Esto implica trabajar no con el modelo conceptual puro, sino con el **modelo relacional ya transformado**. En dicho modelo, las **jerarquías de cuentas y operaciones** fueron unificadas en las tablas Cuenta y Operacion, utilizando atributos discriminadores como Es_ahorro y Tipo_operacion. Como consecuencia, no se han creado clases separadas para CuentaAhorro, CuentaCorriente, Ingreso, Retirada o Transferencia, sino que se ha optado por representar directamente el esquema previo mediante las entidades Cliente, Oficina, Cuenta y Operacion, junto con una clase auxiliar para la clave compuesta de Operacion.

A partir de este planteamiento, el **desarrollo** de esta parte **se organizó en tres fases**. En primer lugar, se verificó que el **entorno Oracle local** seguía disponible y que el **esquema relacional Banquito** estaba correctamente cargado en la base de datos. En segundo lugar, se definieron las **entidades Java y sus anotaciones JPA** para reflejar las tablas y relaciones del diseño relacional previo. Finalmente, se desarrolló una clase de prueba **Test.java** para comprobar el mapeo y ejecutar consultas de ejemplo sobre la base de datos.

4.2. Preparación del entorno Oracle, conexión con el esquema relacional previo y configuración de persistencia

Para esta segunda parte de la práctica **se reutilizó el entorno Oracle** configurado previamente en prácticas anteriores. En lugar de desplegar una nueva instancia de base de datos, se decidió aprovechar la instalación de **Oracle XE** ya existente en la **máquina virtual (VM) TurnKey Core**, gestionada mediante **Docker**. Esta decisión permitió trabajar directamente sobre el esquema relacional Banquito ya implantado. Los comandos necesarios para volver a ejecutar los scripts descritos en la Práctica 2 y dejar operativo dicho entorno se recogen en el [Anexo II](#).

Dado que el **proyecto Java** se ejecutaba **localmente** en el portátil, mientras que **Oracle** residía dentro de un **contenedor Docker alojado en la VM**, fue necesario preparar el entorno de forma que ambos pudieran comunicarse correctamente. El procedimiento seguido fue el siguiente:

- 1. Arranque de la máquina virtual y acceso por SSH.** En primer lugar, se arrancó la VM en VirtualBox y se accedió a ella desde Windows mediante conexión SSH, utilizando el puerto redirigido previamente configurado:

```
ssh -p 2222 root@localhost
```

- 2. Comprobación del estado de Oracle en Docker.** Una vez dentro de la VM, se verificó el estado de los contenedores Docker mediante:

```
root@core ~# docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
dec856212c51   container-registry.oracle.com/database/express:latest   "/bin/bash -c
$ORACL..."    6 weeks ago   Exited (255) 2 weeks ago   0.0.0.0:1521->1521/tcp,
[::]:1521->1521/tcp, 0.0.0.0:5500->5500/tcp, [::]:5500->5500/tcp   oracle-xe
```

Esta comprobación permitió confirmar la existencia del contenedor **oracle-xe** y verificar si se encontraba detenido o en ejecución. En caso de que estuviera parado, se arrancó manualmente con:

```
root@core ~/pr4-oracle# docker start oracle-xe
oracle-xe
```

y se volvió a comprobar su estado con:

```
root@core ~/pr4-oracle# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS          NAMES
dec856212c51   container-registry.oracle.com/database/express:latest   "/bin/bash -c
$ORACL..."    6 weeks ago   Up 5 seconds (health: starting)   0.0.0.0:1521->1521/tcp,
[::]:1521->1521/tcp, 0.0.0.0:5500->5500/tcp, [::]:5500->5500/tcp   oracle-xe
```

- 3. Verificación del acceso al servicio Oracle.** Tras arrancar el contenedor, se comprobó el acceso al servicio Oracle mediante sqlplus, ejecutado dentro del propio contenedor:

```
root@core ~/pr4-oracle# docker exec -it oracle-xe sqlplus
system/oracle123@localhost/XEPDB1

SQL*Plus: Release 21.0.0.0.0 - Production on Wed Apr 8 14:58:45 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Mon Mar 09 2026 18:12:35 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

SQL>
```

4. Comprobación del esquema relacional previo. Una vez establecida la conexión, se verificó que el esquema relacional Banquito seguía cargado en la base de datos. Para ello, se consultaron las tablas principales del diseño relacional desarrollado anteriormente:

```
SQL> SELECT table_name
FROM user_tables
WHERE table_name IN ('CLIENTE', 'OFICINA', 'CUENTA', 'OPERACION', 'SER_TITULAR')
ORDER BY table_name;
```

TABLE_NAME
CLIENTE
CUENTA
OFICINA
OPERACION
SER_TITULAR

Una vez validado el entorno Oracle dentro de la VM, fue necesario **resolver la conectividad entre el portátil y el contenedor Docker**. Para ello, se utilizó una configuración basada en NAT con redirección de puertos en VirtualBox.

El procedimiento seguido fue el siguiente:

1. **Apagar** la máquina virtual.
2. En VirtualBox, **seleccionar la máquina virtual** correspondiente.
3. Acceder a la ruta: **Configuración > Red > Adaptador 1 > Reenvío de puertos**
4. **Añadir una regla de reenvío** con la siguiente configuración:
 - **Name:** oracle
 - **Protocol:** TCP
 - **Host IP:** 127.0.0.1 o vacío
 - **Host Port:** 1521
 - **Guest IP:** vacío
 - **Guest Port:** 1521
5. **Guardar** los cambios.
6. Volver a **arrancar** la máquina virtual.

Gracias a esta configuración, el servicio Oracle quedó accesible desde el portátil como si estuviera escuchando directamente en **localhost:1521**. Para comprobarlo, se ejecutó desde PowerShell la siguiente prueba de conectividad:

```
PS C:\Users\> Test-NetConnection localhost -Port 1521
ComputerName      : localhost
RemoteAddress     : 127.0.0.1
RemotePort        : 1521
InterfaceAlias    : Loopback Pseudo-Interface 1
SourceAddress     : 127.0.0.1
TcpTestSucceeded  : True
```

La prueba devolvió un resultado satisfactorio, por lo que la aplicación Java podía conectarse correctamente a Oracle desde el entorno local.

Seguidamente, se procedió a configurar la persistencia del proyecto. Para ello, se creó el fichero [persistence.xml](#) en la ruta **src/main/resources/META-INF/**. Dado que en esta parte no se pretende generar ni alterar la estructura de la base de datos, sino validar que las entidades Java definidas se corresponden correctamente con el esquema relacional ya existente, se configuró Hibernate en modo de validación mediante la propiedad:

```
<property name="hibernate.hbm2ddl.auto" value="validate"/>
```

En este caso, su función se limita a comprobar la coherencia entre el mapeo JPA y la estructura relacional previamente implantada en Oracle.

4.3. Definición de las entidades Java

Una vez verificado que el esquema relacional Banquito se encontraba accesible desde la aplicación Java y que la conectividad con Oracle era correcta, se procedió a **definir las entidades necesarias** para reflejar en Java la estructura de la base de datos previa. Dado que no se pretende generar un nuevo esquema, sino **establecer la correspondencia** entre el modelo relacional ya existente y el modelo orientado a objetos, las clases se diseñaron **tomando como referencia directa** las tablas y relaciones disponibles en Oracle.

Para ello, se partió de la estructura relacional previamente implementada, compuesta por las relaciones Cliente, Oficina, Cuenta, Operacion y Ser_titular. Y, en consecuencia, **se definieron** las siguientes **clases Java**:

- Cliente
- Oficina
- Cuenta
- Operacion
- OperacionId

Las tres primeras (**Cliente**, **Oficina** y **Cuenta**) se corresponden de forma directa con las tablas del esquema relacional. En cada caso se definieron **atributos equivalentes a las columnas** de Oracle. Así, por ejemplo, **Cliente** recoge los datos identificativos y de contacto de cada titular, **Oficina** representa las sucursales bancarias y **Cuenta** concentra la información de las cuentas bancarias, incluyendo tanto atributos comunes como los que, en el diseño conceptual original, distinguían entre cuentas corrientes y cuentas de ahorro.

Resulta relevante una decisión de diseño adoptada durante el mapeo. Aunque en el modelo conceptual existían las especializaciones CuentaAhorro y CuentaCorriente, en el esquema relacional ambas se integraban en una **única tabla Cuenta**, diferenciándose mediante el **atributo Es_ahorro** y, en su caso, **Tipo_interes** y **Oficina_codigo**. Por este motivo, no se crearon clases Java separadas para cada subtipo, sino una única entidad Cuenta. De forma análoga, tampoco se definieron clases independientes para Ingreso, Retirada o Transferencia, ya que todas las operaciones del sistema se encuentran almacenadas en una **única tabla Operacion**, distinguiéndose por el **atributo Tipo_operacion**.

La clase **Operacion** requirió un tratamiento particular, ya que en el esquema relacional previo esta tabla no se identifica mediante una clave simple, sino por una **clave primaria compuesta** formada por los **atributos Codigo y Cuenta_origen**. Para representar esta situación en JPA se definió una clase auxiliar **OperacionId**, utilizada como **identificador embebido** de la entidad Operacion.

Además de las entidades principales, fue necesario incorporar en las clases Java las relaciones existentes entre tablas. La relación **Ser_titular**, que vincula clientes y cuentas, se representó mediante una **asociación muchos a muchos** entre Cliente y Cuenta. Del mismo modo, se modelaron como relaciones **muchos a uno** los enlaces entre Cuenta y Oficina, así como las referencias existentes en Operacion hacia Cuenta y Oficina. Estas asociaciones permitieron que el modelo Java no se limitara a reflejar atributos aislados, sino también la estructura relacional global del esquema Banquito.

Con esta definición de entidades se obtuvo una representación orientada a objetos del esquema previo. En el **siguiente apartado** se describe con mayor detalle las **anotaciones utilizadas** para mapear las relaciones y la clave compuesta, así como las pruebas realizadas para comprobar el correcto funcionamiento del mapeo. El código fuente completo correspondiente a las clases Java desarrolladas para este mapeo se adjunta en [Anexo I](#).

Con el fin de asegurar que las entidades Java reflejaban fielmente el esquema relacional previo, **se inspeccionó previamente la estructura real de las tablas principales** mediante instrucciones DESC ejecutadas en Oracle. En particular, se revisaron las relaciones Cliente, Oficina, Cuenta, Operacion y Ser_titular, confirmando así los nombres de columnas, tipos de datos y restricciones relevantes para el mapeo. Esta verificación resultó útil para ajustar las anotaciones JPA de atributos, relaciones y claves compuestas. La salida completa de dichas comprobaciones puede consultarse en el [Anexo II](#).

4.4. Mapeo de relaciones y clave compuesta

Una vez definidas las entidades, fue necesario **completar el mapeo objeto/relacional** incorporando las relaciones existentes entre tablas y resolviendo la clave primaria compuesta de la tabla Operacion.

La **primera relación** significativa del esquema relacional previo es **Ser_titular**, que representa la **asociación muchos a muchos entre clientes y cuentas**. En el diseño relacional de Banquito, esta tabla intermedia contiene los atributos Cliente_dni y Cuenta_iban, ambos claves ajenas hacia Cliente y Cuenta, respectivamente. **En el modelo Java**, esta estructura se representó mediante una relación **@ManyToMany** entre las entidades Cliente y Cuenta, utilizando la anotación **@JoinTable** para indicar explícitamente el nombre de la tabla intermedia y las columnas que enlazan con cada una de las entidades. Esta decisión permite reflejar con fidelidad la semántica del diseño previo: un cliente puede ser titular de varias cuentas y una cuenta puede tener varios titulares. Asimismo, **evita modelar Ser_titular como una entidad independiente**, lo cual no aportaba ventajas adicionales en este caso, dado que la tabla no contiene atributos propios más allá de las claves de la relación.

La **segunda relación** relevante es la **asociación entre Cuenta y Oficina**, representada en el esquema relacional mediante el atributo Oficina_codigo de la tabla Cuenta. Esta

columna actúa como clave ajena hacia Oficina(Codigo) y permite reflejar la vinculación entre una cuenta corriente y la oficina bancaria correspondiente. **En el modelo Java**, esta relación se implementó mediante **@ManyToOne** en la entidad Cuenta, acompañada de **@JoinColumn(name = "OFICINA_CODIGO")**. Esta solución resulta natural desde el punto de vista relacional, ya que varias cuentas pueden estar asociadas a una misma oficina, mientras que cada cuenta solo puede referenciar a una oficina concreta. La relación inversa se completó en la entidad Oficina mediante **@OneToMany(mappedBy = "oficina")**, permitiendo navegar desde la oficina hacia el conjunto de cuentas relacionadas.

La **entidad Operacion** presenta un mapeo más complejo, ya que en el esquema relacional previo está **asociada a varias referencias externas**. En primer lugar, Cuenta_origen actúa como clave ajena hacia Cuenta(Iban) y constituye además parte de la clave primaria de la propia tabla. En segundo lugar, Cuenta_destino se utiliza como clave ajena opcional también hacia Cuenta(Iban), necesaria en el caso de las transferencias. Finalmente, Oficina_codigo enlaza con Oficina(Codigo) y se emplea en operaciones de tipo ingreso o retirada. En consecuencia, la entidad Operacion **se definió con tres asociaciones @ManyToOne**: una **hacia Cuenta** para la cuenta origen, otra **hacia Cuenta** para la cuenta destino, y una tercera **hacia Oficina** para la oficina donde se realiza la operación. Esta solución permite reflejar directamente la lógica relacional previa **sin necesidad de introducir subclases específicas** para cada tipo de operación, dado que el propio diseño relacional ya estaba unificado y utilizaba Tipo_operacion como atributo discriminador.

En el esquema relacional previo, la **clave primaria compuesta de la tabla Operacion** está formada por el par **(Codigo, Cuenta_origen)**, lo que implica que una operación no se identifica únicamente por su código, sino por la combinación del código y la cuenta sobre la que se registra. Para representar esta situación en JPA **se definió una clase auxiliar OperacionId**, anotada con **@Embeddable**, que contiene precisamente ambos atributos. Posteriormente, en la entidad Operacion se utilizó **@EmbeddedId** para indicar que el identificador de la entidad es compuesto y se corresponde con la clase OperacionId.

Además, para asociar correctamente el atributo Cuenta_origen de la clave compuesta con la entidad Cuenta, se empleó la anotación **@MapsId("cuentaOrigen")** junto con **@ManyToOne** y **@JoinColumn(name = "CUENTA_ORIGEN")**. Esta configuración permite indicar que una parte del identificador embebido coincide con la clave ajena que referencia a la cuenta origen. De este modo, JPA mantiene sincronizada la relación entre el objeto Operacion y su identificador compuesto, evitando duplicidades o inconsistencias en el modelo.

4.5. Clase de prueba Test.java: inserción, validación y consultas

En esta segunda parte de la práctica **no se ha optado por eliminar previamente todos los datos existentes** en la base de datos, ya que el objetivo consiste en comprobar que el mapeo JPA funciona correctamente sobre el esquema relacional preexistente de Banquito. Por este motivo, las **inserciones realizadas** se plantearon **verificando previamente si los registros de prueba ya existían**. Esta decisión permitió mantener intactos los datos previamente cargados y, al mismo tiempo, **demostrar** que la aplicación Java era capaz de convivir con el contenido ya presente en la base de datos y operar sobre él correctamente. Así

pues, se desarrolló una clase de prueba **Test.java** con el objetivo de centralizar en un único punto la ejecución de las comprobaciones principales.

La ejecución de Test.java **se organizó en cuatro bloques**. En primer lugar, **se inicializa la unidad de persistencia y se establece la conexión con Oracle** mediante EntityManagerFactory y EntityManager. Esta fase constituye la validación inicial del entorno de persistencia, ya que confirma que Hibernate puede leer correctamente la configuración del proyecto y conectarse al esquema Banquito existente. Durante esta primera inicialización **se dejó activada la salida SQL de Hibernate**, con el fin de comprobar visualmente las consultas generadas automáticamente por el framework.

El **segundo bloque** se dedica al **poblado controlado de datos de ejemplo**. Para ello, antes de insertar la oficina, el cliente y las cuentas de prueba, la clase realiza búsquedas por clave primaria utilizando **em.find(...)**. Si la entidad no existe, se crea y se persiste; si ya existe, simplemente se reutiliza. Esta decisión evita duplicidades, permite relanzar el test sin producir errores y, sobre todo, mantiene intactos los datos previamente cargados en la base de datos.

El **tercer bloque registra operaciones bancarias de ejemplo sobre las cuentas de prueba**. En este caso se insertan un ingreso, una retirada y una transferencia, reproduciendo el mismo tipo de operaciones empleadas en la parte 1, pero adaptadas al modelo de la parte 2. Dado que en esta solución no se utilizan subclases específicas para las operaciones, sino una única entidad Operacion apoyada en el atributo tipoOperacion, cada operación se construye **asignando explícitamente el valor correspondiente** (Ingreso, Retirada o Transferencia) **y utilizando una clave compuesta** formada por Codigo y Cuenta_origen. Del mismo modo que en el bloque anterior, antes de persistir cada operación se comprueba si ya existe una operación con esa clave compuesta, lo que evita volver a insertarla en ejecuciones posteriores.

El **cuarto bloque** se dedica a la **ejecución de consultas de ejemplo**. En esta parte se mantuvo el mismo planteamiento seguido en la parte 1 de la práctica, es decir, **formular tres consultas** y ofrecer para cada una de ellas **una versión en Jakarta QL y una versión equivalente en SQL nativo**. Concretamente, se implementaron las siguientes preguntas: el saldo total acumulado de cada cliente, el historial de transferencias superiores a un determinado importe y el saldo medio de las cuentas corrientes por oficina.

Finalmente, la clase de prueba concluye con un **bloque adicional de comprobación de saldos finales sobre las cuentas insertadas** por el propio test. Esta verificación permite confirmar que las operaciones registradas producen el efecto esperado y que la aplicación Java es capaz de consultar el estado actualizado de la base de datos tras la ejecución. En las pruebas realizadas, los resultados obtenidos fueron coherentes con la lógica esperada: tras registrar un ingreso de 1000 euros, una retirada de 200 euros y una transferencia de 300 euros, la cuenta corriente de prueba quedó con un saldo final de 500 euros, mientras que la cuenta de ahorro asociada alcanzó un saldo de 300 euros.

La implementación completa descrita se puede encontrar en [Test.java](#), mientras que la salida íntegra obtenida durante su ejecución puede consultarse en el [Anexo II](#).

4.6. Consultas de ejemplo

Una vez validado el mapeo objeto/relacional, **se definieron tres consultas** de ejemplo con el objetivo de comprobar que el esquema relacional previo de Banquito podía consultarse correctamente desde Java. **Cada consulta se formuló en dos versiones:** una en **JPQL** y otra en **SQL nativo**. En ambos casos se mantuvo la misma lógica, de modo que la comparación entre resultados permitiera comprobar la coherencia entre el modelo de entidades y la estructura física de Oracle.

4.6.1. Consulta C1: saldo total acumulado de cada cliente

La primera consulta calcula el saldo total acumulado de cada cliente a partir del conjunto de cuentas de las que es titular. Técnicamente, esta consulta requiere combinar la información de Cliente, Ser_titular y Cuenta, y aplicar una agregación SUM sobre el saldo actual de las cuentas asociadas a cada cliente.

En JPQL, la consulta se formula a partir de la entidad Cliente y de su relación con Cuenta. La tabla intermedia Ser_titular no aparece en el texto de la consulta porque esa correspondencia ya está resuelta en el mapeo JPA. **En SQL nativo**, en cambio, sí es necesario indicar de forma explícita la unión entre Cliente, Ser_titular y Cuenta, así como las columnas utilizadas en cada enlace.

La comparación entre ambas versiones muestra una diferencia clara. En JPQL, la consulta se redacta a partir de entidades y relaciones definidas en Java. En SQL nativo, la consulta se redacta a partir de tablas y columnas reales de Oracle. En ambos casos, los resultados obtenidos fueron equivalentes, lo que confirma que la relación muchos a muchos entre clientes y cuentas quedó correctamente representada en el modelo Java.

4.6.2. Consulta C2: historial de transferencias superiores a un importe dado

La segunda consulta recupera las operaciones de tipo transferencia cuyo importe supera un determinado umbral. Esta consulta se apoya en la entidad Operacion y en el atributo tipoOperacion.

En JPQL, la consulta se formula directamente sobre la entidad Operacion, aplicando dos condiciones: una sobre tipoOperacion, para seleccionar únicamente las transferencias, y otra sobre cantidad, para limitar el resultado a aquellas cuyo importe supera el valor indicado. Además, la consulta recupera también la cuenta origen y la cuenta destino de cada transferencia, utilizando para ello las referencias incluidas en la propia entidad.

En SQL nativo, la consulta se expresa directamente sobre la tabla Operacion, utilizando las columnas Tipo_operacion y Cantidad como criterios de filtrado. La lógica es la misma que en JPQL, pero la formulación depende de forma explícita de los nombres de columnas definidos en Oracle.

Esta consulta permite comprobar que el mapeo de la tabla Operacion es correcto tanto en sus atributos simples como en sus referencias a cuentas. También permite verificar que la aplicación recupera correctamente transferencias ya existentes en la base de datos y transferencias registradas posteriormente desde la propia clase de prueba.

4.6.3. Consulta C3: saldo medio de las cuentas corrientes por oficina

La tercera consulta calcula el saldo medio de las cuentas corrientes agrupado por oficina. En el esquema relacional previo, las cuentas corrientes y las cuentas de ahorro no se almacenan en tablas distintas, sino en una única tabla Cuenta, diferenciándose mediante el atributo Es_ahorro. Por ello, la consulta debe filtrar las cuentas cuyo valor sea 0 y agrupar después por la oficina asociada.

En JPQL, la consulta se redacta sobre la entidad Cuenta, filtrando por el atributo esAhorro y agrupando por el código de la entidad Oficina relacionada. **En SQL nativo**, la consulta se expresa sobre la tabla Cuenta, utilizando directamente las columnas Es_ahorro y Oficina_codigo, y aplicando la función AVG sobre Saldo_actual.

Por un lado, confirma que el atributo esAhorro permite distinguir correctamente entre cuentas corrientes y cuentas de ahorro dentro del modelo Java. Por otro, comprueba que la relación entre Cuenta y Oficina está bien definida y puede utilizarse en consultas con agrupación y cálculo de medias.

5. Conclusiones

Tras la finalización de esta práctica, se ha podido constatar la **potencia y versatilidad** que ofrece el estándar **Jakarta Persistence** (JPA), y en particular su implementación mediante Hibernate, como herramienta fundamental para cerrar la brecha entre el modelo orientado a objetos y el modelo relacional. El desarrollo del proyecto ha permitido extraer las siguientes conclusiones principales:

- **Abstracción y productividad:** El uso de un framework ORM reduce la complejidad del desarrollo al permitir trabajar con entidades de dominio en lugar de con sintaxis SQL. En esta práctica, la capacidad de generar automáticamente un esquema relacional coherente a partir de anotaciones en Java ha simplificado el proceso de desarrollo y ha garantizado la correspondencia entre el modelo de clases y la base de datos generada.
- **Flexibilidad ante diferentes escenarios:** Se ha comprobado que JPA permite abordar tanto la creación de un sistema desde cero, como la adaptación a esquemas relacionales preexistentes. El uso adecuado de anotaciones como **@Table**, **@Column** o **@JoinTable** resulta clave para ajustar el modelo de objetos a las restricciones del esquema relacional en cada caso.
- **Potencia semántica de JPQL:** El análisis comparativo realizado ha puesto de manifiesto que JPQL no es únicamente un sustituto de SQL, sino un lenguaje de consulta de mayor nivel que permite trabajar directamente sobre el modelo de objetos. La posibilidad de navegar por asociaciones o consultar jerarquías de herencia sin necesidad de definir uniones explícitas aporta una mayor claridad y mantenibilidad al código.
- **Gestión de la identidad y la herencia:** La implementación de elementos como las claves primarias compuestas mediante **@IdClass** o las jerarquías de herencia con la estrategia **SINGLE_TABLE** ha evidenciado la complejidad inherente al mapeo objeto-relacional. En particular, el caso de la entidad *Operacion* ha requerido comprender en detalle cómo JPA gestiona la identidad de los objetos, así como la necesidad de definir correctamente los métodos *equals* y *hashCode* para garantizar su correcto funcionamiento.

Desde un **punto de vista práctico**, durante el desarrollo de la práctica se han encontrado diversas dificultades relacionadas con la configuración del entorno de persistencia y la interpretación de los logs generados por Hibernate. Asimismo, la **comprensión del funcionamiento interno** de ciertas anotaciones, especialmente en el caso de la herencia y las relaciones complejas, ha requerido un proceso de prueba y análisis que ha permitido afianzar los conceptos teóricos vistos en clase.

En definitiva, la **integración de capas de persistencia** basadas en estándares como Jakarta Persistence constituye una herramienta fundamental en el desarrollo de aplicaciones modernas, permitiendo construir sistemas más mantenibles, desacoplados del motor de base de datos subyacente y alineados con los principios de la programación orientada a objetos.

6. Dificultades y lecciones aprendidas

6.1. Correspondencia nomenclatura entre modelo de objetos y esquema relacional

Durante el desarrollo del mapeo objeto-relacional se identificaron diversas dificultades relacionadas con la **generación automática de los nombres de las columnas** en Oracle a partir de los atributos definidos en las clases Java. Estas dificultades no derivan de errores de sintaxis en el código, sino de las **diferencias existentes entre las convenciones de nomenclatura** habituales del paradigma orientado a objetos y el formato físico esperado por el SGBD.

En primer lugar, para los **atributos simples** formados por una única palabra (como *dni* y *telefono*), la traducción realizada por el framework **no presenta conflictos**. Como Oracle no distingue por defecto entre mayúsculas y minúsculas en los identificadores de sus tablas, la base de datos no se ve afectada si en el entorno de Java estos atributos se definen completamente en minúsculas, realizando la correspondencia de forma transparente.

Asimismo, durante la **validación del esquema generado** se detectó una pérdida de formato al procesar atributos complejos. En el diseño lógico desarrollado en la Práctica 2, estos identificadores empleaban **combinaciones de mayúsculas, minúsculas y guiones bajos/snake_case** (como es el caso de *Tipo_operacion*, *Saldo_actual* o *Fecha_creacion*). Al definir estos atributos en Java como una única palabra continuada para adaptarse al estándar del lenguaje (por ejemplo, *tipoooperacion*), Hibernate generaba por defecto columnas físicas con ese mismo texto concatenado. Este comportamiento demuestra que la **delegación automática de la nomenclatura** en el ORM altera la concordancia exacta con el diseño físico previamente establecido.

En resumen, estas dificultades reflejan una limitación importante: **depender exclusivamente** de las estrategias de generación automática de Hibernate **puede no ser adecuado** si se requiere mantener una fidelidad estricta con un diseño de base de datos que utiliza sus propias convenciones de nombres.

Como resultado, se concluye que, **para garantizar** una traducción precisa y coherente con el diseño relacional original, **resulta necesario intervenir** el mapeo por defecto. En esta práctica, la solución adoptada consistió en utilizar la anotación específica **@Column(name = "Nombre_Deseado")** de Jakarta Persistence. De este modo, añadiendo esta directriz justo por encima de la declaración del atributo en Java (por ejemplo, colocando **@Column(name = "Tipo_operacion")** sobre el atributo *tipoooperacion*), se fuerza al motor ORM a construir el esquema físico utilizando el identificador exacto deseado.

6.2. Representación de claves primarias compuestas en entorno orientado a objetos

Durante el desarrollo del mapeo objeto-relacional se identificaron diversas dificultades relacionadas con la **representación de claves primarias compuestas**, de manera específica en la entidad *Operacion*. Estas dificultades no derivan de una limitación en el diseño lógico original, sino de las diferencias fundamentales entre el concepto de identidad relacional (basado en múltiples columnas físicas) y la gestión de la identidad en el paradigma orientado a objetos (que requiere un único objeto identificador).

En primer lugar, se observó que, a nivel de base de datos relacional, la **definición de una clave primaria compuesta** no presenta mayor complejidad que la agrupación de varios atributos. Por ello, la aproximación inicial e intuitiva en el código Java consistió en aplicar la anotación **@Id** simultáneamente sobre los atributos individuales que conformaban dicha clave (el código de la operación y la referencia a la cuenta de origen), asumiendo que el framework concatenaría automáticamente ambos campos para formar la restricción primaria.

Asimismo, durante la **inicialización del contexto de persistencia**, se detectó que esta aproximación directa generaba errores de mapeo y compilación por parte de Hibernate. Este comportamiento se debe a que el estándar de Jakarta Persistence (JPA) exige que cualquier entidad con clave compuesta posea una **representación de identidad** que pueda ser evaluada de forma unitaria en memoria. El motor ORM necesita que dicha identidad **sea serializable y comparable** de forma inequívoca mediante métodos de dispersión, algo que no se consigue simplemente anotando dos atributos inconexos con **@Id** dentro de la propia clase principal.

En resumen, estas dificultades reflejan una limitación importante: el **uso directo y múltiple de la anotación @Id** no es suficiente ni adecuado para que el motor ORM pueda interpretar y gestionar correctamente el ciclo de vida de objetos con dependencias de identificación complejas.

Como resultado, se concluye que, **para garantizar** un mapeo correcto y funcional de entidades con claves compuestas, **resulta necesario encapsular** los atributos identificadores en una estructura auxiliar independiente. En esta práctica, la solución adoptada consistió en la creación de una clase de identidad específica denominada *OperacionId*, la cual implementa la interfaz **Serializable** y define de forma explícita los métodos **equals()** y **hashCode()**. Posteriormente, se utilizó la anotación a nivel de clase **@IdClass(OperacionId.class)** sobre la entidad abstracta *Operacion*, permitiendo a Hibernate comprender la semántica de la clave compuesta y generar correctamente la restricción de clave primaria en el esquema físico.

7. Organización del trabajo y esfuerzos invertidos

Apartados del trabajo realizados	Lucía	Luna
1. Configuración inicial		
Preparación del entorno de desarrollo (IDE, proyecto Java, librerías Hibernate/JPA)	1	0,5
Configuración de las unidades de persistencia y conexión a Oracle (persistence.xml)	0,5	0,5
2. Parte 1: Generación automática		
Desarrollo de clases Java y mapeo ORM completo (atributos, relaciones, claves compuestas y herencia SINGLE_TABLE)	6	1
Desarrollo de <i>Test.java</i> : Población de datos iniciales y simulación de transacciones	2	0
Análisis y validación del esquema relacional generado automáticamente por Hibernate	0,5	0
3. Parte 2: Esquema existente		
Análisis del esquema relacional heredado (tablas, columnas, claves foráneas de la Práctica 2)	0	1
Desarrollo de clases Java y mapeo ORM estricto (@Table, @Column específicos)	1	5
Desarrollo de pruebas para interactuar con la base de datos sin alterar su estructura	0	3
4. Análisis de consultas		
Análisis comparativo entre JPQL y SQL Nativo	1	1
5. Documentación y memoria		
Redacción de la memoria	3	4
Creación de <i>scripts</i>	1	0,5
	16	16,5

8. Bibliografía

Eclipse Foundation. (s.f.). *Jakarta Persistence Specification*. Jakarta EE. Recuperado de <https://jakarta.ee/specifications/persistence/>

Hibernate. (s.f.). *Hibernate ORM 6.2 User Guide*. JBoss / Red Hat. Recuperado de https://docs.jboss.org/hibernate/orm/6.2/userguide/html_single/Hibernate_User_Guide.html

Microsoft. (s.f.). *Java in Visual Studio Code*. Visual Studio Code Documentation. Recuperado de <https://code.visualstudio.com/docs/java/java-tutorial>

Oracle. (s.f.). *Oracle Database JDBC Developer's Guide*. Oracle Help Center. Recuperado de <https://docs.oracle.com/en/database/oracle/oracle-database/>

The Apache Software Foundation. (s.f.). *Maven – Introduction to the POM*. Apache Maven Project. Recuperado de <https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>

9. Anexos

9.1. Anexo I. Código fuente y ficheros de configuración

En este anexo se recoge el contenido completo de los ficheros fuente y de configuración utilizados en la práctica.

9.1.1. Proyecto java-gen-esquema

java-gen-esquema/pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>es.unizar.bd2</groupId>
  <artifactId>p4-jpa</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>3.1.0</version> </dependency>
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>6.2.7.Final</version>
    </dependency>
    <dependency>
      <groupId>com.oracle.database.jdbc</groupId>
      <artifactId>ojdbc11</artifactId>
      <version>23.2.0.0</version>
    </dependency>
  </dependencies>

</project>
```

java-gen-esquema/.../persistence.xml

La contraseña de acceso a la base de datos de Oracle ha sido sustituida por una cadena de asteriscos por motivos de privacidad.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
  version="3.0">

  <persistence-unit name="BanquitoPU" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
```

```

<class>es.unizar.bd2.Cliente</class>
<class>es.unizar.bd2.Oficina</class>
<class>es.unizar.bd2.Cuenta</class>
<class>es.unizar.bd2.CuentaAhorro</class>
<class>es.unizar.bd2.CuentaCorriente</class>
<class>es.unizar.bd2.Operacion</class>
<class>es.unizar.bd2.OperacionTransferencia</class>
<class>es.unizar.bd2.OperacionEfectivo</class>

<properties>
  <property name="hibernate.show_sql" value="true"/>

  <property name="hibernate.hbm2ddl.auto" value="create"/>

  <property name="jakarta.persistence.jdbc.driver" value="oracle.jdbc.driver.OracleDriver"/>
  <property name="jakarta.persistence.jdbc.url"
value="jdbc:oracle:thin:@danae04.unizar.es:1521/barret"/>

  <property name="jakarta.persistence.jdbc.user" value="a871886"/>
  <property name="jakarta.persistence.jdbc.password" value="*****"/>
</properties>
</persistence-unit>

</persistence>

```

java-gen-esquema/.../Cliente.java

```

package es.unizar.bd2;

import jakarta.persistence.*;
import java.util.Set;

@Entity // Le dice a Hibernate que esto será una tabla en la base de datos
public class Cliente {
    @Id // Le dice a Hibernate que el DNI es la clave primaria
    private String dni;

    private String nombre;
    private String apellidos;
    private String email;
    private Integer edad;
    private String direccion;
    private Integer telefono;

    // Relación N:M con Cuenta (tabla ser_titular)
    @ManyToMany
    @JoinTable(
        name = "ser_titular", // Para evitar que le ponga el otro nombre
        joinColumns = @JoinColumn(name = "refCliente"),
        inverseJoinColumns = @JoinColumn(name = "refCuenta")
    )
    private Set<Cuenta> cuentas;

    public Cliente() {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

    public String getDni() {
        return dni;
    }

    public void setDni(String dni) {
        this.dni = dni;
    }
}

```

```
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getApellidos() {
    return apellidos;
}

public void setApellidos(String apellidos) {
    this.apellidos = apellidos;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public Integer getEdad() {
    return edad;
}

public void setEdad(Integer edad) {
    this.edad = edad;
}

public String getDireccion() {
    return direccion;
}

public void setDireccion(String direccion) {
    this.direccion = direccion;
}

public Integer getTelefono() {
    return telefono;
}

public void setTelefono(Integer telefono) {
    this.telefono = telefono;
}

public Set<Cuenta> getCuentas() {
    return cuentas;
}

public void setCuentas(Set<Cuenta> cuentas) {
    this.cuentas = cuentas;
}
}
```

java-gen-esquema/.../Cuenta.java

```
package es.unizar.bd2;

import jakarta.persistence.*;
import java.util.Date;
import java.util.Set;

@Entity // Le dice a Hibernate que esto será una tabla en la base de datos
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) // Le dice a Hibernate que toda la herencia va en una sola
```

```
tabla
@DiscriminatorColumn(name = "Es_ahorro", discriminatorType = DiscriminatorType.INTEGER) // Le dice que Es_ahorro
es la columna que funcionará como discriminador en los tipos de cuentas
public abstract class Cuenta { // abstract porque nunca se creará una cuenta genérica, sino CuentaCorriente o
CuentaAhorro

    @Id // Le dice a Hibernate que el iban es la clave primaria
    private String iban;

    private Integer numero;

    @Column(name = "Fecha_creacion")
    private Date fechacreacion;

    @Column(name = "Saldo_actual")
    private Double saldoactual;

    // Relación N:M con Cliente (Bidireccional)
    // Como en Cliente.java ya está el @ManyToMany, aquí se indica que Cliente es el que manda
    @ManyToMany(mappedBy = "cuentas")
    private Set<Cliente> clientes;

    public Cuenta() {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

    public String getIban() {
        return iban;
    }

    public void setIban(String iban) {
        this.iban = iban;
    }

    public Integer getNumero() {
        return numero;
    }

    public void setNumero(Integer numero) {
        this.numero = numero;
    }

    public Date getFechacreacion() {
        return fechacreacion;
    }

    public void setFechacreacion(Date fechacreacion) {
        this.fechacreacion = fechacreacion;
    }

    public Double getSaldoactual() {
        return saldoactual;
    }

    public void setSaldoactual(Double saldoactual) {
        this.saldoactual = saldoactual;
    }

    public Set<Cliente> getClientes() {
        return clientes;
    }

    public void setClientes(Set<Cliente> clientes) {
        this.clientes = clientes;
    }
}
```

java-gen-esquema/.../CuentaAhorro.java

```
package es.unizar.bd2;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("1") // Le dice a Hibernate: Si Es_ahorro vale '1', entonces es una cuenta ahorro
public class CuentaAhorro extends Cuenta {
    @Column(name = "Tipo_interes")
    private Double tipointeres;

    public CuentaAhorro () {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

    public Double getTipointeres() {
        return tipointeres;
    }

    public void setTipointeres(Double tipointeres) {
        this.tipointeres = tipointeres;
    }
}
```

java-gen-esquema/.../CuentaCorriente.java

```
package es.unizar.bd2;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("0") // Le dice a Hibernate: Si Es_ahorro vale '0', entonces es una cuenta corriente
public class CuentaCorriente extends Cuenta {

    @ManyToOne
    @JoinColumn(name = "refOficina") // Nombre de la clave foránea
    private Oficina oficina;

    public CuentaCorriente () {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

    public Oficina getOficina() {
        return oficina;
    }

    public void setOficina(Oficina oficina) {
        this.oficina = oficina;
    }
}
```

java-gen-esquema/.../Oficina.java

```
package es.unizar.bd2;

import jakarta.persistence.*;

@Entity // Le dice a Hibernate que esto será una tabla en la base de datos
```

```

public class Oficina {
    @Id // Le dice a Hibernate que el codigo es la clave primaria
    private Integer codigo;

    private Integer telefono;
    private String direccion;

    public Oficina () {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

    public Integer getCodigo() {
        return codigo;
    }

    public void setCodigo(Integer codigo) {
        this.codigo = codigo;
    }

    public Integer getTelefono() {
        return telefono;
    }

    public void setTelefono(Integer telefono) {
        this.telefono = telefono;
    }

    public String getDireccion() {
        return direccion;
    }

    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }
}

```

java-gen-esquema/.../Operacion.java

```

package es.unizar.bd2;

import jakarta.persistence.*;
import java.util.Date;

@Entity // Le dice a Hibernate que esto será una tabla en la base de datos
@IdClass(OperacionId.class) // Le dice a JPA que mire esta clase para formar la clave compuesta
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) // Le dice a Hibernate que toda la herencia va en una sola
tabla
@DiscriminatorColumn(name = "Tipo_operacion", discriminatorType = DiscriminatorType.STRING) // Le dice que
Tipo_operacion es la columna que funcionará como discriminador en los tipos de operaciones
public abstract class Operacion {
    @Id // Le dice a Hibernate que el codigo es la clave primaria
    private Integer codigo;

    @Id
    @ManyToOne
    @JoinColumn(name = "refCuentaOrigen") // Nombre de la clave foránea
    private Cuenta cuentaorigen;

    private String concepto;

    @Temporal(TemporalType.TIMESTAMP) // Guarda fecha y hora
    private Date fecha;
    private Double cantidad;

    public Operacion () {

```

```

    // JPA exige que haya un constructor vacío
}

// --- GETTERS Y SETTERS ---

public Integer getCodigo() {
    return codigo;
}

public void setCodigo(Integer codigo) {
    this.codigo = codigo;
}

public String getConcepto() {
    return concepto;
}

public void setConcepto(String concepto) {
    this.concepto = concepto;
}

public Date getFecha() {
    return fecha;
}

public void setFecha(Date fecha) {
    this.fecha = fecha;
}

public Double getCantidad() {
    return cantidad;
}

public void setCantidad(Double cantidad) {
    this.cantidad = cantidad;
}

public Cuenta getCuentaorigen() {
    return cuentaorigen;
}

public void setCuentaorigen(Cuenta cuentaorigen) {
    this.cuentaorigen = cuentaorigen;
}
}

```

java-gen-esquema/.../OperacionEfectivo.java

```

package es.unizar.bd2;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("Efectivo") // Le dice a Hibernate: Si Tipo_operacion es "Efectivo", entonces es una
operación de ingreso o retirada
public class OperacionEfectivo extends Operacion {

    @ManyToOne
    @JoinColumn (name = "refOficina")
    private Oficina oficina;

    public OperacionEfectivo () {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

```

```
public Oficina getOficina() {  
    return oficina;  
}  
  
public void setOficina(Oficina oficina) {  
    this.oficina = oficina;  
}  
}
```

java-gen-esquema/.../OperacionId.java

```
package es.unizar.bd2;  
  
import java.io.Serializable;  
import java.util.Objects;  
  
public class OperacionId implements Serializable { // Representa la clave primaria compuesta de Operacion  
  
    private Integer codigo; // Coincide con el nombre del atributo @Id en Operacion  
    private String cuentaorigen; // Coincide con el nombre del objeto Cuenta en Operacion, pero guarda su ID  
    (IBAN que es String)  
  
    public OperacionId() {}  
  
    public OperacionId(Integer codigo, String cuentaorigen) {  
        this.codigo = codigo;  
        this.cuentaorigen = cuentaorigen;  
    }  
  
    // --- GETTERS Y SETTERS ---  
  
    public Integer getCodigo() {  
        return codigo;  
    }  
  
    public void setCodigo(Integer codigo) {  
        this.codigo = codigo;  
    }  
  
    public String getCuentaorigen() {  
        return cuentaorigen;  
    }  
  
    public void setCuentaorigen(String cuentaorigen) {  
        this.cuentaorigen = cuentaorigen;  
    }  
  
    // JPA exige equals y hashCode para comparar claves compuestas  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (o == null || getClass() != o.getClass()) return false;  
        OperacionId that = (OperacionId) o;  
        return Objects.equals(codigo, that.codigo) && Objects.equals(cuentaorigen, that.cuentaorigen);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(codigo, cuentaorigen);  
    }  
}
```


java-gen-esquema/.../OperacionTransferencia.java

```
package es.unizar.bd2;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("Transferencia") // Le dice a Hibernate: Si Tipo_operacion es "Transferencia", entonces es
una operación de transferencia
public class OperacionTransferencia extends Operacion {

    @ManyToOne
    @JoinColumn (name = "refCuentaDestino")
    private Cuenta cuentadestino;

    public OperacionTransferencia () {
        // JPA exige que haya un constructor vacío
    }

    // --- GETTERS Y SETTERS ---

    public Cuenta getCuentadestino() {
        return cuentadestino;
    }

    public void setCuentadestino(Cuenta cuentadestino) {
        this.cuentadestino = cuentadestino;
    }
}
```

java-gen-esquema/.../Test.java

```
package es.unizar.bd2;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import jakarta.persistence.Query;
import java.util.Date;
import java.util.HashSet;
import java.util.List;

public class Test {
    public static void main(String[] args) {
        System.out.println("Iniciando Hibernate y conectando a la base de datos...");
        EntityManagerFactory emf = null;
        EntityManager em = null;

        try {
            emf = Persistence.createEntityManagerFactory("BanquitoPU");
            em = emf.createEntityManager();

            // =====
            // BLOQUE 1: POBLAR LA BASE DE DATOS
            // =====
            em.getTransaction().begin();
            System.out.println("1. Creando datos iniciales (Oficinas, Clientes y Cuentas)...");

            // 1.1 Crear Oficina
            Oficina ofi = new Oficina();
            ofi.setCodigo(1);
            ofi.setDireccion("Cajero Principal Centro");
            ofi.setTelefono(900112233);
            em.persist(ofi);

            // 1.2 Crear Cliente
            Cliente cli = new Cliente();
```

```

cli.setDni("1111111H");
cli.setNombre("Usuario");
cli.setApellidos("De Prueba");
cli.setEdad(30);
cli.setDireccion("Avenida Principal, 1");
cli.setEmail("usuario.prueba@banco.com");
cli.setCuentas(new HashSet<>()); // Inicializamos el Set
em.persist(cli);

// 1.3 Crear Cuenta Corriente
CuentaCorriente cc = new CuentaCorriente();
cc.setIban("ES91000011122233334444");
cc.setNumero(1111);
cc.setFechaCreacion(new Date());
cc.setSaldoActual(1000.0);
cc.setOficina(ofi);
em.persist(cc);

// 1.4 Crear Cuenta Ahorro (Destino de la transferencia)
CuentaAhorro ca = new CuentaAhorro();
ca.setIban("ES9100005555666677778888");
ca.setNumero(5555);
ca.setFechaCreacion(new Date());
ca.setSaldoActual(500.0);
ca.setTipointeres(2.5);
em.persist(ca);

// 1.5 Asociar Cliente a Cuentas (N:M ser_titular)
cli.getCuentas().add(cc);
cli.getCuentas().add(ca);

em.getTransaction().commit();
System.out.println("    -> Datos iniciales guardados correctamente.\n");

// =====
// BLOQUE 2: OPERACIONES BANCARIAS
// =====
em.getTransaction().begin();
System.out.println("2. Registrando operaciones de Efectivo y Transferencia...\n");

// 2.1 Operación Efectivo (Cajero - Retirada de 200€)
OperacionEfectivo opEfectivo = new OperacionEfectivo();
opEfectivo.setCodigo(1);
opEfectivo.setConcepto("Cajero");
opEfectivo.setFecha(new Date());
opEfectivo.setCantidad(-200.0);
opEfectivo.setCuentaorigen(cc);
opEfectivo.setOficina(ofi);
em.persist(opEfectivo);

// Actualizar saldo
cc.setSaldoActual(cc.getSaldoActual() - 200.0);

// 2.2 Operación Transferencia (Regalo - 300€)
OperacionTransferencia opTransf = new OperacionTransferencia();
opTransf.setCodigo(2);
opTransf.setConcepto("Regalo");
opTransf.setFecha(new Date());
opTransf.setCantidad(300.0);
opTransf.setCuentaorigen(cc);
opTransf.setCuentadestino(ca);
em.persist(opTransf);

// Actualizar saldos origen y destino
cc.setSaldoActual(cc.getSaldoActual() - 300.0);
ca.setSaldoActual(ca.getSaldoActual() + 300.0);

em.getTransaction().commit();
System.out.println("    -> Operaciones registradas y saldos actualizados.\n");

// =====

```

```

// BLOQUE 3: CONSULTAS JPQL vs SQL NATIVO (Comparativa para La Memoria)
// =====
System.out.println("3. Ejecutando consultas de prueba...\n");

// --- CONSULTA 1: JOIN N:M (Saldo total por cliente) ---
System.out.println("--- C1: Saldo total acumulado de cada cliente ---");

// JPQL: Navega por Los objetos directamente usando La colección 'cuentas'
System.out.println(" * Usando JPQL:");
String jpql1 = "SELECT c.nombre, SUM(cu.saldoactual) FROM Cliente c JOIN c.cuentas cu GROUP BY
c.nombre";
List<Object[]> resJpql1 = em.createQuery(jpql1).getResultList();
for (Object[] r : resJpql1) System.out.println("  Cliente: " + r[0] + " | Saldo Total: " + r[1] +
"€");

// SQL NATIVO: Requiere hacer Los JOIN explícitos con La tabla intermedia (ser_titular)
System.out.println(" * Usando SQL Nativo:");
String sql1 = "SELECT c.nombre, SUM(cu.Saldo_actual) FROM Cliente c " +
"JOIN ser_titular st ON c.dni = st.refCliente " +
"JOIN Cuenta cu ON st.refCuenta = cu.iban GROUP BY c.nombre";
List<Object[]> resSql1 = em.createNativeQuery(sql1).getResultList();
for (Object[] r : resSql1) System.out.println("  Cliente: " + r[0] + " | Saldo Total: " + r[1] +
"€");
System.out.println();

// --- CONSULTA 2: POLIMORFISMO Y HERENCIA (Buscar transferencias) ---
System.out.println("--- C2: Historial de Transferencias mayores a X importe ---");
double limite = 100.0;

// JPQL: Consulta directamente a La clase hija 'OperacionTransferencia'. Hibernate deduce el
discriminador.
System.out.println(" * Usando JPQL:");
String jpql2 = "SELECT o.concepto, o.cantidad, o.cuentaorigen.iban, o.cuentadestino.iban " +
"FROM OperacionTransferencia o WHERE o.cantidad > :limite";
Query qJpql2 = em.createQuery(jpql2);
qJpql2.setParameter("limite", limite);
List<Object[]> resJpql2 = qJpql2.getResultList();
for (Object[] r : resJpql2) System.out.println("  " + r[0] + ": " + r[1] + "€ (De: " + r[2] + " A: "
+ r[3] + ")");

// SQL NATIVO: AL usar SINGLE_TABLE, hay que filtrar manualmente por La columna discriminadora.
System.out.println(" * Usando SQL Nativo:");
String sql2 = "SELECT concepto, cantidad, refCuentaOrigen, refCuentaDestino " +
"FROM Operacion WHERE Tipo_operacion = 'Transferencia' AND cantidad > ?";
Query qSql2 = em.createNativeQuery(sql2);
qSql2.setParameter(1, limite);
List<Object[]> resSql2 = qSql2.getResultList();
for (Object[] r : resSql2) System.out.println("  " + r[0] + ": " + r[1] + "€ (De: " + r[2] + " A: "
+ r[3] + ")");
System.out.println();

// --- CONSULTA 3: AGRUPACIÓN Y CLASES HIJAS (Saldo medio Cuentas Corrientes por Oficina) ---
System.out.println("--- C3: Saldo medio de las cuentas corrientes por código de oficina ---");

// JPQL: Consultamos La clase CuentaCorriente y navegamos a La oficina sin hacer JOINS explícitos
System.out.println(" * Usando JPQL:");
String jpql3 = "SELECT c.oficina.codigo, AVG(c.saldoactual) FROM CuentaCorriente c GROUP BY
c.oficina.codigo";
List<Object[]> resJpql3 = em.createQuery(jpql3).getResultList();
for (Object[] r : resJpql3) System.out.println("  Oficina: " + r[0] + " | Saldo Medio: " + r[1] +
"€");

// SQL NATIVO: Usamos La tabla Cuenta y filtramos por La columna discriminadora 'Es_ahorro' = 0
System.out.println(" * Usando SQL Nativo:");
String sql3 = "SELECT refOficina, AVG(Saldo_actual) FROM Cuenta WHERE Es_ahorro = 0 GROUP BY
refOficina";
List<Object[]> resSql3 = em.createNativeQuery(sql3).getResultList();
for (Object[] r : resSql3) System.out.println("  Oficina: " + r[0] + " | Saldo Medio: " + r[1] +
"€");

```

```

    } catch (Exception e) {
        System.err.println("Ha ocurrido un error durante la ejecución:");
        e.printStackTrace();
        if (em != null && em.getTransaction().isActive()) {
            em.getTransaction().rollback();
        }
    } finally {
        if (em != null) em.close();
        if (emf != null) emf.close();
        System.out.println("\nEjecución finalizada y conexión cerrada.");
    }
}
}

```

9.1.2. Proyecto java-previo-esquema

java-previo-esquema/pom.xml

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>es.unizar.bd2</groupId>
  <artifactId>java-previo-esquema</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>6.4.4.Final</version>
    </dependency>

    <dependency>
      <groupId>jakarta.persistence</groupId>
      <artifactId>jakarta.persistence-api</artifactId>
      <version>3.1.0</version>
    </dependency>

    <dependency>
      <groupId>com.oracle.database.jdbc</groupId>
      <artifactId>ojdbc11</artifactId>
      <version>23.3.0.23.09</version>
    </dependency>

    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-simple</artifactId>
      <version>2.0.12</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.1.0</version>
      </plugin>
    </plugins>
  </build>
</project>

```

```

        </plugin>
    </plugins>
</build>
</project>

```

java-previo-esquema/.../persistence.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence
https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
    version="3.0">

    <persistence-unit name="banquito-previo" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <class>es.unizar.bd2.Cliente</class>
        <class>es.unizar.bd2.Oficina</class>
        <class>es.unizar.bd2.Cuenta</class>
        <class>es.unizar.bd2.Operacion</class>

        <properties>
            <property name="hibernate.show_sql" value="true" />
            <property name="hibernate.format_sql" value="true" />
            <property name="jakarta.persistence.jdbc.driver" value="oracle.jdbc.OracleDriver" />

            <property name="jakarta.persistence.jdbc.url" value="jdbc:oracle:thin:@localhost:1521/XEPDB1" />
            <property name="jakarta.persistence.jdbc.user" value="system" />
            <property name="jakarta.persistence.jdbc.password" value="oracle123" />
            <property name="hibernate.hbm2ddl.auto" value="validate" />

            <property name="hibernate.dialect" value="org.hibernate.dialect.OracleDialect" />
        </properties>
    </persistence-unit>
</persistence>

```

java-previo-esquema/.../Cliente.java

```

package es.unizar.bd2;

import jakarta.persistence.*;
import java.util.LinkedHashSet;
import java.util.Objects;
import java.util.Set;

@Entity
@Table(name = "CLIENTE")
public class Cliente {

    @Id
    @Column(name = "DNI", length = 9, nullable = false)
    private String dni;

    @Column(name = "NOMBRE", length = 50, nullable = false)
    private String nombre;

    @Column(name = "EMAIL", length = 50)
    private String email;

    @Column(name = "APELLIDOS", length = 50, nullable = false)
    private String apellidos;

    @Column(name = "EDAD", precision = 3, nullable = false)

```

```
private Integer edad;

@Column(name = "DIRECCION", length = 100, nullable = false)
private String direccion;

@Column(name = "TELEFONO", precision = 9, nullable = false)
private Long telefono;

@ManyToMany(mappedBy = "titulares")
private Set<Cuenta> cuentas = new LinkedHashSet<>();

public Cliente() {
}

public String getDni() {
    return dni;
}

public void setDni(String dni) {
    this.dni = dni;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getApellidos() {
    return apellidos;
}

public void setApellidos(String apellidos) {
    this.apellidos = apellidos;
}

public Integer getEdad() {
    return edad;
}

public void setEdad(Integer edad) {
    this.edad = edad;
}

public String getDireccion() {
    return direccion;
}

public void setDireccion(String direccion) {
    this.direccion = direccion;
}

public Long getTelefono() {
    return telefono;
}

public void setTelefono(Long telefono) {
    this.telefono = telefono;
}

public Set<Cuenta> getCuentas() {
```

```

        return cuentas;
    }

    public void setCuentas(Set<Cuenta> cuentas) {
        this.cuentas = cuentas;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (!(o instanceof Cliente cliente))
            return false;
        return Objects.equals(dni, cliente.dni);
    }

    @Override
    public int hashCode() {
        return Objects.hash(dni);
    }
}

```

java-previo-esquema/.../Cuenta.java

```

package es.unizar.bd2;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.util.Date;
import java.util.LinkedHashSet;
import java.util.Objects;
import java.util.Set;

@Entity
@Table(name = "CUENTA")
public class Cuenta {

    @Id
    @Column(name = "IBAN", length = 24, nullable = false)
    private String iban;

    @Column(name = "NUMERO", precision = 20, scale = 0, nullable = false)
    private BigDecimal numero;

    @Temporal(TemporalType.DATE)
    @Column(name = "FECHA_CREACION", nullable = false)
    private Date fechaCreacion;

    @Column(name = "SALDO_ACTUAL", precision = 10, scale = 2, nullable = false)
    private BigDecimal saldoActual;

    @Column(name = "ES_AHORRO", precision = 1, nullable = false)
    private Integer esAhorro;

    @Column(name = "TIPO_INTERES", precision = 5, scale = 2)
    private BigDecimal tipoInteres;

    @ManyToOne
    @JoinColumn(name = "OFICINA_CODIGO")
    private Oficina oficina;

    @ManyToMany
    @JoinTable(name = "SER_TITULAR", joinColumns = @JoinColumn(name = "CUENTA_IBAN"), inverseJoinColumns =
    @JoinColumn(name = "CLIENTE_DNI"))
    private Set<Cliente> titulares = new LinkedHashSet<>();

    @OneToMany(mappedBy = "cuentaOrigen")
}

```

```
private Set<Operacion> operacionesOrigen = new LinkedHashSet<>();

@OneToMany(mappedBy = "cuentaDestino")
private Set<Operacion> operacionesDestino = new LinkedHashSet<>();

public Cuenta() {
}

public String getIban() {
    return iban;
}

public void setIban(String iban) {
    this.iban = iban;
}

public BigDecimal getNumero() {
    return numero;
}

public void setNumero(BigDecimal numero) {
    this.numero = numero;
}

public Date getFechaCreacion() {
    return fechaCreacion;
}

public void setFechaCreacion(Date fechaCreacion) {
    this.fechaCreacion = fechaCreacion;
}

public BigDecimal getSaldoActual() {
    return saldoActual;
}

public void setSaldoActual(BigDecimal saldoActual) {
    this.saldoActual = saldoActual;
}

public Integer getEsAhorro() {
    return esAhorro;
}

public void setEsAhorro(Integer esAhorro) {
    this.esAhorro = esAhorro;
}

public BigDecimal getTipoInteres() {
    return tipoInteres;
}

public void setTipoInteres(BigDecimal tipoInteres) {
    this.tipoInteres = tipoInteres;
}

public Oficina getOficina() {
    return oficina;
}

public void setOficina(Oficina oficina) {
    this.oficina = oficina;
}

public Set<Cliente> getTitulares() {
    return titulares;
}

public void setTitulares(Set<Cliente> titulares) {
    this.titulares = titulares;
}
```



```

public Set<Operacion> getOperacionesOrigen() {
    return operacionesOrigen;
}

public void setOperacionesOrigen(Set<Operacion> operacionesOrigen) {
    this.operacionesOrigen = operacionesOrigen;
}

public Set<Operacion> getOperacionesDestino() {
    return operacionesDestino;
}

public void setOperacionesDestino(Set<Operacion> operacionesDestino) {
    this.operacionesDestino = operacionesDestino;
}

public boolean esCuentaAhorro() {
    return esAhorro != null && esAhorro == 1;
}

public boolean esCuentaCorriente() {
    return esAhorro != null && esAhorro == 0;
}

@Override
public boolean equals(Object o) {
    if (this == o)
        return true;
    if (!(o instanceof Cuenta cuenta))
        return false;
    return Objects.equals(iban, cuenta.iban);
}

@Override
public int hashCode() {
    return Objects.hash(iban);
}
}

```

java-previo-esquema/.../Oficina.java

```

package es.unizar.bd2;

import jakarta.persistence.*;
import java.util.LinkedHashSet;
import java.util.Objects;
import java.util.Set;

@Entity
@Table(name = "OFICINA")
public class Oficina {

    @Id
    @Column(name = "CODIGO", precision = 5, nullable = false)
    private Integer codigo;

    @Column(name = "TELEFONO", precision = 9, nullable = false)
    private Long telefono;

    @Column(name = "DIRECCION", length = 100, nullable = false)
    private String direccion;

    @OneToMany(mappedBy = "oficina")
    private Set<Cuenta> cuentas = new LinkedHashSet<>();

    @OneToMany(mappedBy = "oficina")

```

```

private Set<Operacion> operaciones = new LinkedHashSet<>();

public Oficina() {
}

public Integer getCodigo() {
    return codigo;
}

public void setCodigo(Integer codigo) {
    this.codigo = codigo;
}

public Long getTelefono() {
    return telefono;
}

public void setTelefono(Long telefono) {
    this.telefono = telefono;
}

public String getDireccion() {
    return direccion;
}

public void setDireccion(String direccion) {
    this.direccion = direccion;
}

public Set<Cuenta> getCuentas() {
    return cuentas;
}

public void setCuentas(Set<Cuenta> cuentas) {
    this.cuentas = cuentas;
}

public Set<Operacion> getOperaciones() {
    return operaciones;
}

public void setOperaciones(Set<Operacion> operaciones) {
    this.operaciones = operaciones;
}

@Override
public boolean equals(Object o) {
    if (this == o)
        return true;
    if (!(o instanceof Oficina oficina))
        return false;
    return Objects.equals(codigo, oficina.codigo);
}

@Override
public int hashCode() {
    return Objects.hash(codigo);
}
}

```

java-previo-esquema/.../Operacion.java

```

package es.unizar.bd2;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.util.Date;

```

```
import java.util.Objects;

@Entity
@Table(name = "OPERACION")
public class Operacion {

    @EmbeddedId
    private OperacionId id;

    @Column(name = "CONCEPTO", length = 250)
    private String concepto;

    @Temporal(TemporalType.DATE)
    @Column(name = "FECHA", nullable = false)
    private Date fecha;

    @Column(name = "CANTIDAD", precision = 10, scale = 2, nullable = false)
    private BigDecimal cantidad;

    @Column(name = "TIPO_OPERACION", length = 13, nullable = false)
    private String tipoOperacion;

    @MapsId("cuentaOrigen")
    @ManyToOne
    @JoinColumn(name = "CUENTA_ORIGEN", nullable = false)
    private Cuenta cuentaOrigen;

    @ManyToOne
    @JoinColumn(name = "OFICINA_CODIGO")
    private Oficina oficina;

    @ManyToOne
    @JoinColumn(name = "CUENTA_DESTINO")
    private Cuenta cuentaDestino;

    public Operacion() {
    }

    public OperacionId getId() {
        return id;
    }

    public void setId(OperacionId id) {
        this.id = id;
    }

    public String getConcepto() {
        return concepto;
    }

    public void setConcepto(String concepto) {
        this.concepto = concepto;
    }

    public Date getFecha() {
        return fecha;
    }

    public void setFecha(Date fecha) {
        this.fecha = fecha;
    }

    public BigDecimal getCantidad() {
        return cantidad;
    }

    public void setCantidad(BigDecimal cantidad) {
        this.cantidad = cantidad;
    }

    public String getTipoOperacion() {
```

```

        return tipoOperacion;
    }

    public void setTipoOperacion(String tipoOperacion) {
        this.tipoOperacion = tipoOperacion;
    }

    public Cuenta getCuentaOrigen() {
        return cuentaOrigen;
    }

    public void setCuentaOrigen(Cuenta cuentaOrigen) {
        this.cuentaOrigen = cuentaOrigen;
    }

    public Oficina getOficina() {
        return oficina;
    }

    public void setOficina(Oficina oficina) {
        this.oficina = oficina;
    }

    public Cuenta getCuentaDestino() {
        return cuentaDestino;
    }

    public void setCuentaDestino(Cuenta cuentaDestino) {
        this.cuentaDestino = cuentaDestino;
    }

    public boolean esTransferencia() {
        return "Transferencia".equalsIgnoreCase(tipoOperacion);
    }

    public boolean esIngreso() {
        return "Ingreso".equalsIgnoreCase(tipoOperacion);
    }

    public boolean esRetirada() {
        return "Retirada".equalsIgnoreCase(tipoOperacion);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (!(o instanceof Operacion operacion))
            return false;
        return Objects.equals(id, operacion.id);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}

```

java-previo-esquema/.../OperacionId.java

```

package es.unizar.bd2;

import jakarta.persistence.Column;
import jakarta.persistence.Embeddable;
import java.io.Serializable;
import java.util.Objects;

```

```
@Embeddable
public class OperacionId implements Serializable {

    @Column(name = "CODIGO", precision = 8, nullable = false)
    private Integer codigo;

    @Column(name = "CUENTA_ORIGEN", length = 24, nullable = false)
    private String cuentaOrigen;

    public OperacionId() {
    }

    public OperacionId(Integer codigo, String cuentaOrigen) {
        this.codigo = codigo;
        this.cuentaOrigen = cuentaOrigen;
    }

    public Integer getCodigo() {
        return codigo;
    }

    public void setCodigo(Integer codigo) {
        this.codigo = codigo;
    }

    public String getCuentaOrigen() {
        return cuentaOrigen;
    }

    public void setCuentaOrigen(String cuentaOrigen) {
        this.cuentaOrigen = cuentaOrigen;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o)
            return true;
        if (!(o instanceof OperacionId that))
            return false;
        return Objects.equals(codigo, that.codigo)
            && Objects.equals(cuentaOrigen, that.cuentaOrigen);
    }

    @Override
    public int hashCode() {
        return Objects.hash(codigo, cuentaOrigen);
    }
}
```

java-previo-esquema/.../Test.java

```
package es.unizar.bd2;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import jakarta.persistence.Query;

import java.math.BigDecimal;
import java.util.Date;
import java.util.HashSet;
import java.util.List;

public class Test {

    public static void main(String[] args) {
        System.out.println("Iniciando Hibernate y conectando a la base de datos...");
    }
}
```

```

EntityManagerFactory emf = null;
EntityManager em = null;

try {
    emf = Persistence.createEntityManagerFactory("banquito-previo");
    em = emf.createEntityManager();

    // =====
    // BLOQUE 1: POBLADO CONTROLADO DE DATOS DE EJEMPLO
    // =====
    System.out.println("=====");
    System.out.println("1. POBLADO CONTROLADO DE DATOS DE EJEMPLO");
    System.out.println("=====");

    em.getTransaction().begin();

    // 1.1 Crear o recuperar Oficina
    Oficina ofi = em.find(Oficina.class, 99);
    if (ofi == null) {
        ofi = new Oficina();
        ofi.setCodigo(99);
        ofi.setDireccion("Oficina de Pruebas JPA");
        ofi.setTelefono(900112233L);
        em.persist(ofi);
        System.out.println("    -> Oficina creada.");
    } else {
        System.out.println("    -> Oficina ya existente.");
    }

    // 1.2 Crear o recuperar Cliente
    Cliente cli = em.find(Cliente.class, "1111111H");
    if (cli == null) {
        cli = new Cliente();
        cli.setDni("1111111H");
        cli.setNombre("Usuario");
        cli.setApellidos("De Prueba");
        cli.setEdad(30);
        cli.setDireccion("Avenida Principal, 1");
        cli.setEmail("usuario.prueba@banco.com");
        cli.setTelefono(600123123L);
        cli.setCuentas(new HashSet<>());
        em.persist(cli);
        System.out.println("    -> Cliente creado.");
    } else {
        System.out.println("    -> Cliente ya existente.");
    }

    // 1.3 Crear o recuperar Cuenta Corriente
    Cuenta cc = em.find(Cuenta.class, "ES91990011112223334444");
    if (cc == null) {
        cc = new Cuenta();
        cc.setIban("ES91990011112223334444");
        cc.setNumero(new BigDecimal("11112223334444"));
        cc.setFechaCreacion(new Date());
        cc.setSaldoActual(new BigDecimal("0.00"));
        cc.setEsAhorro(0);
        cc.setTipoInteres(null);
        cc.setOficina(ofi);
        cc.setTitulares(new HashSet<>());
        em.persist(cc);
        System.out.println("    -> Cuenta corriente creada.");
    } else {
        System.out.println("    -> Cuenta corriente ya existente.");
    }

    // 1.4 Crear o recuperar Cuenta Ahorro
    Cuenta ca = em.find(Cuenta.class, "ES9199005555666677778888");
    if (ca == null) {
        ca = new Cuenta();
        ca.setIban("ES9199005555666677778888");
        ca.setNumero(new BigDecimal("5555666677778888"));
    }
}

```

```

        ca.setFechaCreacion(new Date());
        ca.setSaldoActual(new BigDecimal("0.00"));
        ca.setEsAhorro(1);
        ca.setTipoInteres(new BigDecimal("2.50"));
        ca.setOficina(null);
        ca.setTitulares(new HashSet<>());
        em.persist(ca);
        System.out.println("    -> Cuenta ahorro creada.");
    } else {
        System.out.println("    -> Cuenta ahorro ya existente.");
    }
}

// 1.5 Asociar Cliente a Cuentas (N:M ser_titular)
if (!cc.getTitulares().contains(cli)) {
    cc.getTitulares().add(cli);
}
if (!ca.getTitulares().contains(cli)) {
    ca.getTitulares().add(cli);
}
if (!cli.getCuentas().contains(cc)) {
    cli.getCuentas().add(cc);
}
if (!cli.getCuentas().contains(ca)) {
    cli.getCuentas().add(ca);
}

em.getTransaction().commit();
System.out.println("    -> Datos de ejemplo verificados/preparados correctamente.\n");

// =====
// BLOQUE 2: REGISTRO DE OPERACIONES BANCARIAS
// =====
System.out.println("=====");
System.out.println("2. REGISTRO DE OPERACIONES BANCARIAS");
System.out.println("=====");

em.getTransaction().begin();

// 2.1 Ingreso de 1000 euros
OperacionId idIngreso = new OperacionId(1001, cc.getIban());
Operacion opIngreso = em.find(Operacion.class, idIngreso);
if (opIngreso == null) {
    opIngreso = new Operacion();
    opIngreso.setId(idIngreso);
    opIngreso.setConcepto("Nomina JPA");
    opIngreso.setFecha(new Date());
    opIngreso.setCantidad(new BigDecimal("1000.00"));
    opIngreso.setTipoOperacion("Ingreso");
    opIngreso.setCuentaOrigen(cc);
    opIngreso.setOficina(ofi);
    opIngreso.setCuentaDestino(null);
    em.persist(opIngreso);
    System.out.println("    -> Ingreso registrado.");
} else {
    System.out.println("    -> Ingreso ya existente.");
}

// 2.2 Retirada de 200 euros
OperacionId idRetirada = new OperacionId(1002, cc.getIban());
Operacion opRetirada = em.find(Operacion.class, idRetirada);
if (opRetirada == null) {
    opRetirada = new Operacion();
    opRetirada.setId(idRetirada);
    opRetirada.setConcepto("Cajero JPA");
    opRetirada.setFecha(new Date());
    opRetirada.setCantidad(new BigDecimal("200.00"));
    opRetirada.setTipoOperacion("Retirada");
    opRetirada.setCuentaOrigen(cc);
    opRetirada.setOficina(ofi);
    opRetirada.setCuentaDestino(null);
    em.persist(opRetirada);
}

```

```

        System.out.println("    -> Retirada registrada.");
    } else {
        System.out.println("    -> Retirada ya existente.");
    }

    // 2.3 Transferencia de 300 euros a La cuenta de ahorro
    OperacionId idTransferencia = new OperacionId(1003, cc.getIban());
    Operacion opTransferencia = em.find(Operacion.class, idTransferencia);
    if (opTransferencia == null) {
        opTransferencia = new Operacion();
        opTransferencia.setId(idTransferencia);
        opTransferencia.setConcepto("Transferencia JPA");
        opTransferencia.setFecha(new Date());
        opTransferencia.setCantidad(new BigDecimal("300.00"));
        opTransferencia.setTipoOperacion("Transferencia");
        opTransferencia.setCuentaOrigen(cc);
        opTransferencia.setOficina(null);
        opTransferencia.setCuentaDestino(ca);
        em.persist(opTransferencia);
        System.out.println("    -> Transferencia registrada.");
    } else {
        System.out.println("    -> Transferencia ya existente.");
    }

    em.getTransaction().commit();
    System.out.println("    -> Operaciones bancarias registradas correctamente.\n");

    // Se limpia el contexto para releer el estado real actualizado desde BD
    em.clear();

    // =====
    // BLOQUE 3: CONSULTAS DE EJEMPLO
    // =====
    System.out.println("=====");
    System.out.println("3. CONSULTAS DE EJEMPLO");
    System.out.println("=====\\n");

    // -----
    // C1. SALDO TOTAL ACUMULADO DE CADA CLIENTE
    // -----
    System.out.println("-----");
    System.out.println("C1. SALDO TOTAL ACUMULADO DE CADA CLIENTE");
    System.out.println("-----");

    // JPQL
    System.out.println(" * Usando JPQL:");
    String jpql1 = "SELECT c.nombre, SUM(cu.saldoActual) " +
        "FROM Cliente c JOIN c.cuentas cu " +
        "GROUP BY c.nombre";
    List<Object[]> resJpql1 = em.createQuery(jpql1).getResultList();
    for (Object[] r : resJpql1) {
        System.out.println("    Cliente: " + r[0] + " | Saldo Total: " + r[1] + " euros");
    }

    // SQL NATIVO
    System.out.println(" * Usando SQL Nativo:");
    String sql1 = "SELECT c.NOMBRE, SUM(cu.SALDO_ACTUAL) " +
        "FROM CLIENTE c " +
        "JOIN SER_TITULAR st ON c.DNI = st.CLIENTE_DNI " +
        "JOIN CUENTA cu ON st.CUENTA_IBAN = cu.IBAN " +
        "GROUP BY c.NOMBRE";
    List<Object[]> resSql1 = em.createNativeQuery(sql1).getResultList();
    for (Object[] r : resSql1) {
        System.out.println("    Cliente: " + r[0] + " | Saldo Total: " + r[1] + " euros");
    }
    System.out.println();

    // -----
    // C2. HISTORIAL DE TRANSFERENCIAS MAYORES A UN IMPORTE DADO
    // -----
    System.out.println("-----");

```



```

System.out.println("C2. HISTORIAL DE TRANSFERENCIAS MAYORES A UN IMPORTE DADO");
System.out.println("-----");

BigDecimal limite = new BigDecimal("100.00");

// JPQL
System.out.println(" * Usando JPQL:");
String jpql2 = "SELECT o.concepto, o.cantidad, o.cuentaOrigen.iban, o.cuentaDestino.iban " +
    "FROM Operacion o " +
    "WHERE o.tipoOperacion = :tipo AND o.cantidad > :limite";
Query qJpql2 = em.createQuery(jpql2);
qJpql2.setParameter("tipo", "Transferencia");
qJpql2.setParameter("limite", limite);
List<Object[]> resJpql2 = qJpql2.getResultList();
for (Object[] r : resJpql2) {
    System.out.println("    " + r[0] + ": " + r[1] + " euros (De: " + r[2] + " A: " + r[3] + ")");
}

// SQL NATIVO
System.out.println(" * Usando SQL Nativo:");
String sql2 = "SELECT CONCEPTO, CANTIDAD, CUENTA_ORIGEN, CUENTA_DESTINO " +
    "FROM OPERACION " +
    "WHERE TIPO_OPERACION = 'Transferencia' AND CANTIDAD > ?";
Query qSql2 = em.createNativeQuery(sql2);
qSql2.setParameter(1, limite);
List<Object[]> resSql2 = qSql2.getResultList();
for (Object[] r : resSql2) {
    System.out.println("    " + r[0] + ": " + r[1] + " euros (De: " + r[2] + " A: " + r[3] + ")");
}
System.out.println();

// -----
// C3. SALDO MEDIO DE LAS CUENTAS CORRIENTES POR OFICINA
// -----
System.out.println("-----");
System.out.println("C3. SALDO MEDIO DE LAS CUENTAS CORRIENTES POR OFICINA");
System.out.println("-----");

// JPQL
System.out.println(" * Usando JPQL:");
String jpql3 = "SELECT c.oficina.codigo, AVG(c.saldoActual) " +
    "FROM Cuenta c " +
    "WHERE c.esAhorro = 0 " +
    "GROUP BY c.oficina.codigo";
List<Object[]> resJpql3 = em.createQuery(jpql3).getResultList();
for (Object[] r : resJpql3) {
    System.out.println("    Oficina: " + r[0] + " | Saldo Medio: " + r[1] + " euros");
}

// SQL NATIVO
System.out.println(" * Usando SQL Nativo:");
String sql3 = "SELECT OFICINA_CODIGO, AVG(SALDO_ACTUAL) " +
    "FROM CUENTA " +
    "WHERE ES_AHORRO = 0 " +
    "GROUP BY OFICINA_CODIGO";
List<Object[]> resSql3 = em.createNativeQuery(sql3).getResultList();
for (Object[] r : resSql3) {
    System.out.println("    Oficina: " + r[0] + " | Saldo Medio: " + r[1] + " euros");
}
System.out.println();

// =====
// BLOQUE 4: COMPROBACION DE SALDOS FINALES
// =====
System.out.println("=====");
System.out.println("4. COMPROBACION DE SALDOS FINALES");
System.out.println("=====");

Cuenta cuentaCorrienteFinal = em.find(Cuenta.class, "ES91990011112223334444");
Cuenta cuentaAhorroFinal = em.find(Cuenta.class, "ES9199005555666677778888");

```

```

        System.out.println("    Cuenta corriente: " + cuentaCorrienteFinal.getIban() +
            " | Saldo: " + cuentaCorrienteFinal.getSaldoActual() + " euros");
        System.out.println("    Cuenta ahorro: " + cuentaAhorroFinal.getIban() +
            " | Saldo: " + cuentaAhorroFinal.getSaldoActual() + " euros");

    } catch (Exception e) {
        System.err.println("Ha ocurrido un error durante la ejecución:");
        e.printStackTrace();
        if (em != null && em.getTransaction().isActive()) {
            em.getTransaction().rollback();
        }
    } finally {
        if (em != null)
            em.close();
        if (emf != null)
            emf.close();
        System.out.println("\nEjecución finalizada y conexión cerrada.");
    }
}
}
}

```

9.2. Anexo II. Salidas de ejecución y comprobaciones

En este anexo se presentan las salidas completas obtenidas durante la ejecución de los scripts y pruebas más relevantes de la práctica.

9.2.1. Salida de creación del esquema relacional Banquito en Oracle

```

root@core ~/pr4-oracle# docker cp ~/pr4-oracle/bd-bancaria-oracle.sql oracle-xe:/tmp/
p/
Successfully copied 8.19kB to oracle-xe:/tmp/

root@core ~/pr4-oracle# docker exec -it oracle-xe sqlplus system/oracle123@localhost/XEPDB1
@/tmp/bd-bancaria-oracle.sql

SQL*Plus: Release 21.0.0.0.0 - Production on Wed Apr 8 15:07:57 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Wed Apr 08 2026 14:58:45 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
PL/SQL procedure successfully completed.
Table created.
Table created.
Table created.
Table created.
Table created.
Trigger created.
Trigger created.
Sequence created.
Procedure created.
PL/SQL procedure successfully completed.

SQL>

```

9.2.2. Salida de pruebas del esquema relacional Banquito

```
root@core ~/pr4-oracle# docker cp ~/pr4-oracle/bd-bancaria-pruebas-oracle.sql oracle-xe:/tmp/
Successfully copied 8.19kB to oracle-xe:/tmp/

root@core ~/pr4-oracle# docker exec -it oracle-xe sqlplus system/oracle123@localhost/XEPDB1
@/tmp/bd-bancaria-pruebas-oracle.sql

SQL*Plus: Release 21.0.0.0.0 - Production on Wed Apr 8 15:08:27 2026
Version 21.3.0.0.0

Copyright (c) 1982, 2021, Oracle. All rights reserved.

Last Successful login time: Wed Apr 08 2026 15:07:57 +00:00

Connected to:
Oracle Database 21c Express Edition Release 21.0.0.0.0 - Production
Version 21.3.0.0.0

Session altered.

=====
0. LIMPIEZA DE DATOS PREVIOS
=====
0 rows deleted.
0 rows deleted.
0 rows deleted.
0 rows deleted.
0 rows deleted.
Commit complete.
Datos anteriores eliminados correctamente.

=====
1. INSERCIÓN DE DATOS BASE (DEBEN FUNCIONAR CORRECTAMENTE)
=====
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
1 row created.
Commit complete.
Datos base insertados.

=====
2. PRUEBAS DE RESTRICCIONES (AQUI DEBEN SALIR ERRORES ORA-)
=====
> Prueba 2.1: Intentar insertar un DNI falso (Falla por Regex)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008575) violated

> Prueba 2.2: Intentar insertar un menor de edad (Falla por CHECK)
INSERT INTO Cliente (Dni, Nombre, Apellidos, Email, Edad, Direccion, Telefono)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008574) violated

> Prueba 2.3: Cuenta de Ahorro sin tipo de interes (Falla por CHECK de jerarquia)
INSERT INTO Cuenta (Iban, Numero, Es_ahorro, Tipo_interes, Oficina_codigo, Saldo_actual)
*
ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008586) violated

> Prueba 2.4: Transferencia sin Cuenta Destino (Falla por CHECK de operacion)
INSERT INTO Operacion (Codigo, Concepto, Cantidad, Tipo_operacion, Cuenta_origen, Oficina_codigo, Cuenta_destino)
*
```

```

ERROR at line 1:
ORA-02290: check constraint (SYSTEM.SYS_C008596) violated

=====
3. PRUEBAS DE TRIGGERS Y OPERACIONES (DEBEN FUNCIONAR)
=====
> Hacemos un Ingreso de 1000 euros en la cuenta de Ana
1 row created.

> Hacemos una Retirada de 200 euros en la cuenta de Ana
1 row created.

> Ana hace una transferencia de 300 euros a Luis
1 row created.

> ESTADO DE SALDOS ESPERADO:
> Ana empezo con 0 + 1000 - 200 - 300 = 500 euros.
> Luis empezo con 0 + 300 transferidos = 300 euros.
IBAN                SALDO_ACTUAL
-----
ES91000011112223334444      500
ES910000555566667778888      300

=====
4. PRUEBA DE DISPARADOR EN CADENA (FONDOS INSUFICIENTES)
=====
> Ana intenta sacar 600 euros pero solo tiene 500.
> El trigger restara el saldo y Oracle parara la insercion por el CHECK de saldo >= 0.
VALUES (seq_operacion.NEXTVAL, 'Compra TV', 600, 'Retirada', 'ES910000111122233334444', 1, NULL)
*

ERROR at line 2:
ORA-02290: check constraint (SYSTEM.SYS_C008584) violated
ORA-06512: at "SYSTEM.TRG_ACTUALIZA_SALDO", line 7
ORA-04088: error during execution of trigger 'SYSTEM.TRG_ACTUALIZA_SALDO'

> Comprobamos que el saldo de Ana sigue intacto en 500 euros

IBAN                SALDO_ACTUAL
-----
ES91000011112223334444      500

=====
5. PRUEBA DEL PROCEDIMIENTO DE INTERESES
=====
> Ejecutamos el pago de intereses manual (Liquidacion de la cuenta de ahorro de Luis)
PL/SQL procedure successfully completed.

> Comprobamos las operaciones. Deberia aparecer un nuevo Ingreso automatico para Luis
CODIGO CONCEPTO                CANTIDAD TIPO_OPERACIO
-----
2 Nomina                        1000 Ingreso
3 Cajero                        200 Retirada
4 Regalo                        300 Transferencia
6 Liquidacion de intereses mensuales .62 Ingreso

> Comprobamos que el saldo de Luis ha subido (Tenia 300, y se le ha sumado su interes)
IBAN                SALDO_ACTUAL
-----
ES910000555566667778888      300.62

Commit complete.

PRUEBAS FINALIZADAS.

SQL>

```

9.2.3. Verificación de tablas disponibles en Oracle

```
SQL> DESC CLIENTE;
Name                               Null?    Type
-----
DNI                                NOT NULL VARCHAR2(9)
NOMBRE                             NOT NULL VARCHAR2(50)
EMAIL                             VARCHAR2(50)
APELLIDOS                          NOT NULL VARCHAR2(50)
EDAD                               NOT NULL NUMBER(3)
DIRECCION                          NOT NULL VARCHAR2(100)
TELEFONO                           NOT NULL NUMBER(9)

SQL> DESC OFICINA;
Name                               Null?    Type
-----
CODIGO                             NOT NULL NUMBER(5)
TELEFONO                           NOT NULL NUMBER(9)
DIRECCION                          NOT NULL VARCHAR2(100)

SQL> DESC CUENTA;
Name                               Null?    Type
-----
IBAN                               NOT NULL VARCHAR2(24)
NUMERO                             NOT NULL NUMBER(20)
FECHA_CREACION                     NOT NULL DATE
SALDO_ACTUAL                       NOT NULL NUMBER(10,2)
ES_AHORRO                          NOT NULL NUMBER(1)
TIPO_INTERES                       NUMBER(5,2)
OFICINA_CODIGO                     NUMBER(5)

SQL> DESC OPERACION;
Name                               Null?    Type
-----
CODIGO                             NOT NULL NUMBER(8)
CONCEPTO                         VARCHAR2(250)
FECHA                              NOT NULL DATE
CANTIDAD                           NOT NULL NUMBER(10,2)
TIPO_OPERACION                     NOT NULL VARCHAR2(13)
CUENTA_ORIGEN                      NOT NULL VARCHAR2(24)
OFICINA_CODIGO                     NUMBER(5)
CUENTA_DESTINO                     VARCHAR2(24)

SQL> DESC SER_TITULAR;
Name                               Null?    Type
-----
CLIENTE_DNI                        NOT NULL VARCHAR2(9)
CUENTA_IBAN                        NOT NULL VARCHAR2(24)
```

9.2.4. Salida de ejecución de Test.java en java-gen-esquema

```
PS D:\CUARTO\BASES2\PRACTICAS\PRACTICA4\JPA> &
'C:\Users\luvam\.vscode\extensions\redhat.java-1.53.0-win32-x64\jre\21.0.10-win32-x86_64\bin\java.exe'
'@C:\Users\luvam\AppData\Local\Temp\cp_c214a54kwkcf7n0i9oe4fsuk2.argfile' 'es.unizar.bd2.Test'
Iniciando Hibernate y conectando a la base de datos...
abr 08, 2026 5:59:01 P. M. org.hibernate.jpa.internal.util.LogHelper logPersistenceUnitInformation
INFO: HHH000204: Processing PersistenceUnitInfo [name: BanquitoPU]
abr 08, 2026 5:59:01 P. M. org.hibernate.Version logVersion
INFO: HHH000412: Hibernate ORM core version 6.2.7.Final
abr 08, 2026 5:59:01 P. M. org.hibernate.cfg.Environment <clinit>
INFO: HHH000406: Using bytecode reflection optimizer
abr 08, 2026 5:59:02 P. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
configure
WARN: HHH10001002: Using built-in connection pool (not intended for production use)
abr 08, 2026 5:59:02 P. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001005: Loaded JDBC driver class: oracle.jdbc.driver.OracleDriver
```

```

abr 08, 2026 5:59:02 P. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001012: Connecting with JDBC URL [jdbc:oracle:thin:@danae04.unizar.es:1521/barret]
abr 08, 2026 5:59:02 P. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=a871886}
abr 08, 2026 5:59:02 P. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001003: Autocommit mode: false
abr 08, 2026 5:59:02 P. M.
org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH10001115: Connection pool size: 20 (min=1)
abr 08, 2026 5:59:03 P. M. org.hibernate.bytecode.internal.BytecodeProviderInitiator buildBytecodeProvider
INFO: HHH000021: Bytecode provider name : bytebuddy
abr 08, 2026 5:59:03 P. M. org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator
initiateService
INFO: HHH000490: Using JtaPlatform implementation:
[org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
Hibernate: drop table Cliente cascade constraints
abr 08, 2026 5:59:03 P. M.
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess
[org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess@6b3d9c38]
for (non-JTA) DDL execution was not in auto-commit mode; the Connection 'local transaction' will be committed and
the Connection will be set into auto-commit mode.
Hibernate: drop table Cuenta cascade constraints
Hibernate: drop table Oficina cascade constraints
Hibernate: drop table Operacion cascade constraints
Hibernate: drop table ser_titular cascade constraints
Hibernate: create table Cliente (edad number(10,0), telefono number(10,0), apellidos varchar2(255 char), direccion
varchar2(255 char), dni varchar2(255 char) not null, email varchar2(255 char), nombre varchar2(255 char), primary
key (dni))
abr 08, 2026 5:59:04 P. M.
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess
[org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess@b3857e2]
for (non-JTA) DDL execution was not in auto-commit mode; the Connection 'local transaction' will be committed and
the Connection will be set into auto-commit mode.
Hibernate: create table Cuenta (Es_ahorro number(10,0) not null, Saldo_actual float(53), Tipo_interes float(53),
numero number(10,0), refOficina number(10,0), Fecha_creacion timestamp(6), iban varchar2(255 char) not null,
primary key (iban))
Hibernate: create table Oficina (codigo number(10,0) not null, telefono number(10,0), direccion varchar2(255
char), primary key (codigo))
Hibernate: create table Operacion (cantidad float(53), codigo number(10,0) not null, refOficina number(10,0),
fecha timestamp(6), Tipo_operacion varchar2(31 char) not null, concepto varchar2(255 char), refCuentaDestino
varchar2(255 char), refCuentaOrigen varchar2(255 char) not null, primary key (codigo, refCuentaOrigen))
Hibernate: create table ser_titular (refCliente varchar2(255 char) not null, refCuenta varchar2(255 char) not
null, primary key (refCliente, refCuenta))
Hibernate: alter table Cuenta add constraint FKc5c566hxwv6ue2ufx5duwslp foreign key (refOficina) references
Oficina
Hibernate: alter table Operacion add constraint FK2ca48g4k2e8x0kbtsal8hwhp2 foreign key (refCuentaOrigen)
references Cuenta
Hibernate: alter table Operacion add constraint FKodc3s0hmd9k2txskl3w6n0q5j foreign key (refOficina) references
Oficina
Hibernate: alter table Operacion add constraint FKetyykdchcwsj97uk5mv503o7w foreign key (refCuentaDestino)
references Cuenta
Hibernate: alter table ser_titular add constraint FKble52c3cs2ivqgt44773jhxnw foreign key (refCuenta) references
Cuenta
Hibernate: alter table ser_titular add constraint FKa8cm0vgvkhhxpq5rtb5okpyvf foreign key (refCliente) references
Cliente
1. Creando datos iniciales (Oficinas, Clientes y Cuentas)...
Hibernate: insert into Oficina (direccion,telefono,codigo) values (?, ?, ?)
Hibernate: insert into Cliente (apellidos,direccion,edad,email,nombre,telefono,dni) values (?, ?, ?, ?, ?, ?, ?)
Hibernate: insert into Cuenta (Fecha_creacion,numero,Saldo_actual,refOficina,Es_ahorro,iban) values (?, ?, ?, ?, 0, ?)
Hibernate: insert into Cuenta (Fecha_creacion,numero,Saldo_actual,Tipo_interes,Es_ahorro,iban) values
(?, ?, ?, ?, 1, ?)
Hibernate: insert into ser_titular (refCliente,refCuenta) values (?, ?)
Hibernate: insert into ser_titular (refCliente,refCuenta) values (?, ?)
-> Datos iniciales guardados correctamente.

2. Registrando operaciones de Efectivo y Transferencia...

```

```

Hibernate: insert into Operacion (cantidad,concepto,fecha,refOficina,Tipo_operacion,codigo,refCuentaOrigen) values
(?,?,?,,'Efectivo',?,?)
Hibernate: insert into Operacion (cantidad,concepto,fecha,refCuentaDestino,Tipo_operacion,codigo,refCuentaOrigen)
values (?,?,?,,'Transferencia',?,?)
Hibernate: update Cuenta set Fecha_creacion=?,numero=?,Saldo_actual=?,refOficina=? where iban=?
Hibernate: update Cuenta set Fecha_creacion=?,numero=?,Saldo_actual=?,Tipo_interes=? where iban=?
    -> Operaciones registradas y saldos actualizados.

3. Ejecutando consultas de prueba...

--- C1: Saldo total acumulado de cada cliente ---
* Usando JPQL:
Hibernate: select c1_0.nombre,sum(c2_1.Saldo_actual) from Cliente c1_0 join (ser_titular c2_0 join Cuenta c2_1 on
c2_1.iban=c2_0.refCuenta) on c1_0.dni=c2_0.refCliente group by c1_0.nombre
    Cliente: Usuario | Saldo Total: 1300.0?
* Usando SQL Nativo:
Hibernate: SELECT c.nombre, SUM(cu.Saldo_actual) FROM Cliente c JOIN ser_titular st ON c.dni = st.refCliente JOIN
Cuenta cu ON st.refCuenta = cu.iban GROUP BY c.nombre
    Cliente: Usuario | Saldo Total: 1300?

--- C2: Historial de Transferencias mayores a X importe ---
* Usando JPQL:
Hibernate: select o1_0.concepto,o1_0.cantidad,o1_0.refCuentaOrigen,o1_0.refCuentaDestino from Operacion o1_0 where
o1_0.cantidad>? and o1_0.Tipo_operacion='Transferencia'
    Regalo: 300.0? (De: ES910000111122233334444 A: ES9100005555666677778888)
* Usando SQL Nativo:
Hibernate: SELECT concepto, cantidad, refCuentaOrigen, refCuentaDestino FROM Operacion WHERE Tipo_operacion =
'Transferencia' AND cantidad > ?
    Regalo: 300.0? (De: ES910000111122233334444 A: ES9100005555666677778888)

--- C3: Saldo medio de las cuentas corrientes por código de oficina ---
* Usando JPQL:
Hibernate: select c1_0.refOficina,avg(c1_0.Saldo_actual) from Cuenta c1_0 where c1_0.Es_ahorro=0 group by
c1_0.refOficina
    Oficina: 1 | Saldo Medio: 500.0?
* Usando SQL Nativo:
Hibernate: SELECT refOficina, AVG(Saldo_actual) FROM Cuenta WHERE Es_ahorro = 0 GROUP BY refOficina
    Oficina: 1 | Saldo Medio: 500?
abr 08, 2026 5:59:05 P. M.
org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:oracle:thin:@danae04.unizar.es:1521/barret]

Ejecución finalizada y conexión cerrada.

```

9.2.5. Salida de ejecución de Test.java en java-previo-esquema

```

PS D:\BASES2\Practicas\practica4\code> d:; cd 'd:\BASES2\Practicas\practica4\code'; &
'C:\Users\lunaz\.vscode\extensions\redhat.java-1.53.0-win32-x64\jre\21.0.10-win32-x86_64\bin\java.exe'
'@C:\Users\lunaz\AppData\Local\Temp\cp_2pwxthsoc9eic09w8lp16ggr2.argfile' 'es.unizar.bd2.Test'
Iniciando Hibernate y conectando a la base de datos...
abr 09, 2026 11:34:52 A. M. org.hibernate.jpa.internal.util.LogHelper logPersistenceUnitInformation
INFO: HHH000204: Processing PersistenceUnitInfo [name: banquito-previo]
abr 09, 2026 11:34:52 A. M. org.hibernate.Version logVersion
INFO: HHH000412: Hibernate ORM core version 6.4.4.Final
abr 09, 2026 11:34:52 A. M. org.hibernate.cache.internal.RegionFactoryInitiator initiateService
INFO: HHH00026: Second-level cache disabled
abr 09, 2026 11:34:52 A. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
configure
WARN: HHH10001002: Using built-in connection pool (not intended for production use)
abr 09, 2026 11:34:53 A. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001005: Loaded JDBC driver class: oracle.jdbc.OracleDriver
abr 09, 2026 11:34:53 A. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001012: Connecting with JDBC URL [jdbc:oracle:thin:@localhost:1521/XEPDB1]
abr 09, 2026 11:34:53 A. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001001: Connection properties: {password=****, user=system}

```

```

abr 09, 2026 11:34:53 A. M. org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl
buildCreator
INFO: HHH10001003: Autocommit mode: false
abr 09, 2026 11:34:53 A. M.
org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PooledConnections <init>
INFO: HHH10001115: Connection pool size: 20 (min=1)
abr 09, 2026 11:34:54 A. M. org.hibernate.engine.jdbc.dialect.internal.DialectFactoryImpl constructDialect
WARN: HHH9000025: OracleDialect does not need to be specified explicitly using 'hibernate.dialect' (remove the
property setting and it will be selected by default)
abr 09, 2026 11:34:56 A. M. org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator
initiateService
INFO: HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform
integration)
abr 09, 2026 11:34:56 A. M.
org.hibernate.resource.transaction.backend.jdbc.internal.DdlTransactionIsolatorNonJtaImpl getIsolatedConnection
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess
[org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionProviderJdbcConnectionAccess@6598caab
] for (non-JTA) DDL execution was not in auto-commit mode; the Connection 'local transaction' will be committed
and the Connection will be set into auto-commit mode.
=====
1. POBLADO CONTROLADO DE DATOS DE EJEMPLO
=====
Hibernate:
select
  o1_0.CODIGO,
  o1_0.DIRECCION,
  o1_0.TELEFONO
from
  OFICINA o1_0
where
  o1_0.CODIGO=?
-> Oficina ya existente.
Hibernate:
select
  c1_0.DNI,
  c1_0.APELLIDOS,
  c1_0.DIRECCION,
  c1_0.EDAD,
  c1_0.EMAIL,
  c1_0.NOMBRE,
  c1_0.TELEFONO
from
  CLIENTE c1_0
where
  c1_0.DNI=?
-> Cliente ya existente.
Hibernate:
select
  c1_0.IBAN,
  c1_0.ES_AHORRO,
  c1_0.FECHA_CREACION,
  c1_0.NUMERO,
  o1_0.CODIGO,
  o1_0.DIRECCION,
  o1_0.TELEFONO,
  c1_0.SALDO_ACTUAL,
  c1_0.TIPO_INTERES
from
  CUENTA c1_0
left join
  OFICINA o1_0
    on o1_0.CODIGO=c1_0.OFICINA_CODIGO
where
  c1_0.IBAN=?
-> Cuenta corriente ya existente.
Hibernate:
select
  c1_0.IBAN,
  c1_0.ES_AHORRO,
  c1_0.FECHA_CREACION,
  c1_0.NUMERO,

```



```

        o1_0.CODIGO,
        o1_0.DIRECCION,
        o1_0.TELEFONO,
        c1_0.SALDO_ACTUAL,
        c1_0.TIPO_INTERES
    from
        CUENTA c1_0
    left join
        OFICINA o1_0
        on o1_0.CODIGO=c1_0.OFICINA_CODIGO
    where
        c1_0.IBAN=?
    -> Cuenta ahorro ya existente.
Hibernate:
    select
        t1_0.CUENTA_IBAN,
        t1_1.DNI,
        t1_1.APELLIDOS,
        t1_1.DIRECCION,
        t1_1.EDAD,
        t1_1.EMAIL,
        t1_1.NOMBRE,
        t1_1.TELEFONO
    from
        SER_TITULAR t1_0
    join
        CLIENTE t1_1
        on t1_1.DNI=t1_0.CLIENTE_DNI
    where
        t1_0.CUENTA_IBAN=?
Hibernate:
    select
        t1_0.CUENTA_IBAN,
        t1_1.DNI,
        t1_1.APELLIDOS,
        t1_1.DIRECCION,
        t1_1.EDAD,
        t1_1.EMAIL,
        t1_1.NOMBRE,
        t1_1.TELEFONO
    from
        SER_TITULAR t1_0
    join
        CLIENTE t1_1
        on t1_1.DNI=t1_0.CLIENTE_DNI
    where
        t1_0.CUENTA_IBAN=?
Hibernate:
    select
        c1_0.CLIENTE_DNI,
        c1_1.IBAN,
        c1_1.ES_AHORRO,
        c1_1.FECHA_CREACION,
        c1_1.NUMERO,
        o1_0.CODIGO,
        o1_0.DIRECCION,
        o1_0.TELEFONO,
        c1_1.SALDO_ACTUAL,
        c1_1.TIPO_INTERES
    from
        SER_TITULAR c1_0
    join
        CUENTA c1_1
        on c1_1.IBAN=c1_0.CUENTA_IBAN
    left join
        OFICINA o1_0
        on o1_0.CODIGO=c1_1.OFICINA_CODIGO
    where
        c1_0.CLIENTE_DNI=?
    -> Datos de ejemplo verificados/preparados correctamente.

```

=====

2. REGISTRO DE OPERACIONES BANCARIAS

=====

Hibernate:

```

select
    o1_0.CODIGO,
    o1_0.CUENTA_ORIGEN,
    o1_0.CANTIDAD,
    o1_0.CONCEPTO,
    cd1_0.IBAN,
    cd1_0.ES_AHORRO,
    cd1_0.FECHA_CREACION,
    cd1_0.NUMERO,
    o2_0.CODIGO,
    o2_0.DIRECCION,
    o2_0.TELEFONO,
    cd1_0.SALDO_ACTUAL,
    cd1_0.TIPO_INTERES,
    co1_0.IBAN,
    co1_0.ES_AHORRO,
    co1_0.FECHA_CREACION,
    co1_0.NUMERO,
    o3_0.CODIGO,
    o3_0.DIRECCION,
    o3_0.TELEFONO,
    co1_0.SALDO_ACTUAL,
    co1_0.TIPO_INTERES,
    o1_0.FECHA,
    o4_0.CODIGO,
    o4_0.DIRECCION,
    o4_0.TELEFONO,
    o1_0.TIPO_OPERACION
from
    OPERACION o1_0
left join
    CUENTA cd1_0
        on cd1_0.IBAN=o1_0.CUENTA_DESTINO
left join
    OFICINA o2_0
        on o2_0.CODIGO=cd1_0.OFICINA_CODIGO
join
    CUENTA co1_0
        on co1_0.IBAN=o1_0.CUENTA_ORIGEN
left join
    OFICINA o3_0
        on o3_0.CODIGO=co1_0.OFICINA_CODIGO
left join
    OFICINA o4_0
        on o4_0.CODIGO=o1_0.OFICINA_CODIGO
where
    (
        o1_0.CODIGO, o1_0.CUENTA_ORIGEN
    ) in ((?, ?))
-> Ingreso ya existente.

```

Hibernate:

```

select
    o1_0.CODIGO,
    o1_0.CUENTA_ORIGEN,
    o1_0.CANTIDAD,
    o1_0.CONCEPTO,
    cd1_0.IBAN,
    cd1_0.ES_AHORRO,
    cd1_0.FECHA_CREACION,
    cd1_0.NUMERO,
    o2_0.CODIGO,
    o2_0.DIRECCION,
    o2_0.TELEFONO,
    cd1_0.SALDO_ACTUAL,
    cd1_0.TIPO_INTERES,
    co1_0.IBAN,
    co1_0.ES_AHORRO,

```

```

        co1_0.FECHA_CREACION,
        co1_0.NUMERO,
        o3_0.CODIGO,
        o3_0.DIRECCION,
        o3_0.TELEFONO,
        co1_0.SALDO_ACTUAL,
        co1_0.TIPO_INTERES,
        o1_0.FECHA,
        o4_0.CODIGO,
        o4_0.DIRECCION,
        o4_0.TELEFONO,
        o1_0.TIPO_OPERACION
    from
        OPERACION o1_0
    left join
        CUENTA cd1_0
            on cd1_0.IBAN=o1_0.CUENTA_DESTINO
    left join
        OFICINA o2_0
            on o2_0.CODIGO=cd1_0.OFICINA_CODIGO
    join
        CUENTA co1_0
            on co1_0.IBAN=o1_0.CUENTA_ORIGEN
    left join
        OFICINA o3_0
            on o3_0.CODIGO=co1_0.OFICINA_CODIGO
    left join
        OFICINA o4_0
            on o4_0.CODIGO=o1_0.OFICINA_CODIGO
    where
        (
            o1_0.CODIGO, o1_0.CUENTA_ORIGEN
        ) in (?, ?))
    -> Retirada ya existente.
Hibernate:
    select
        o1_0.CODIGO,
        o1_0.CUENTA_ORIGEN,
        o1_0.CANTIDAD,
        o1_0.CONCEPTO,
        cd1_0.IBAN,
        cd1_0.ES_AHORRO,
        cd1_0.FECHA_CREACION,
        cd1_0.NUMERO,
        o2_0.CODIGO,
        o2_0.DIRECCION,
        o2_0.TELEFONO,
        cd1_0.SALDO_ACTUAL,
        cd1_0.TIPO_INTERES,
        co1_0.IBAN,
        co1_0.ES_AHORRO,
        co1_0.FECHA_CREACION,
        co1_0.NUMERO,
        o3_0.CODIGO,
        o3_0.DIRECCION,
        o3_0.TELEFONO,
        co1_0.SALDO_ACTUAL,
        co1_0.TIPO_INTERES,
        o1_0.FECHA,
        o4_0.CODIGO,
        o4_0.DIRECCION,
        o4_0.TELEFONO,
        o1_0.TIPO_OPERACION
    from
        OPERACION o1_0
    left join
        CUENTA cd1_0
            on cd1_0.IBAN=o1_0.CUENTA_DESTINO
    left join
        OFICINA o2_0
            on o2_0.CODIGO=cd1_0.OFICINA_CODIGO

```

```

join
    CUENTA co1_0
    on co1_0.IBAN=o1_0.CUENTA_ORIGEN
left join
    OFICINA o3_0
    on o3_0.CODIGO=co1_0.OFICINA_CODIGO
left join
    OFICINA o4_0
    on o4_0.CODIGO=o1_0.OFICINA_CODIGO
where
    (
        o1_0.CODIGO, o1_0.CUENTA_ORIGEN
    ) in ((?, ?))
-> Transferencia ya existente.
-> Operaciones bancarias registradas correctamente.

```

3. CONSULTAS DE EJEMPLO

C1. SALDO TOTAL ACUMULADO DE CADA CLIENTE

* Usando JPQL:

Hibernate:

```

select
    c1_0.NOMBRE,
    sum(c2_1.SALDO_ACTUAL)
from
    CLIENTE c1_0
join
    SER_TITULAR c2_0
    on c1_0.DNI=c2_0.CLIENTE_DNI
join
    CUENTA c2_1
    on c2_1.IBAN=c2_0.CUENTA_IBAN
group by
    c1_0.NOMBRE
Cliente: Ana | Saldo Total: 500 euros
Cliente: Luis | Saldo Total: 301.25 euros
Cliente: Usuario | Saldo Total: 800 euros

```

* Usando SQL Nativo:

Hibernate:

```

SELECT
    c.NOMBRE,
    SUM(cu.SALDO_ACTUAL)
FROM
    CLIENTE c
JOIN
    SER_TITULAR st
    ON c.DNI = st.CLIENTE_DNI
JOIN
    CUENTA cu
    ON st.CUENTA_IBAN = cu.IBAN
GROUP BY
    c.NOMBRE
Cliente: Ana | Saldo Total: 500 euros
Cliente: Luis | Saldo Total: 301.25 euros
Cliente: Usuario | Saldo Total: 800 euros

```

C2. HISTORIAL DE TRANSFERENCIAS MAYORES A UN IMPORTE DADO

* Usando JPQL:

Hibernate:

```

select
    o1_0.CONCEPTO,
    o1_0.CANTIDAD,
    o1_0.CUENTA_ORIGEN,
    o1_0.CUENTA_DESTINO
from

```

```

OPERACION o1_0
where
  o1_0.TIPO_OPERACION=?
  and o1_0.CANTIDAD>?
Regalo: 300 euros (De: ES910000111122233334444 A: ES9100005555666677778888)
Transferencia JPA: 300 euros (De: ES919900111122233334444 A: ES9199005555666677778888)
* Usando SQL Nativo:
Hibernate:
SELECT
  CONCEPTO,
  CANTIDAD,
  CUENTA_ORIGEN,
  CUENTA_DESTINO
FROM
  OPERACION
WHERE
  TIPO_OPERACION = 'Transferencia'
  AND CANTIDAD > ?
Regalo: 300 euros (De: ES910000111122233334444 A: ES9100005555666677778888)
Transferencia JPA: 300 euros (De: ES919900111122233334444 A: ES9199005555666677778888)

```

C3. SALDO MEDIO DE LAS CUENTAS CORRIENTES POR OFICINA

```

* Usando JPQL:
Hibernate:
select
  c1_0.OFICINA_CODIGO,
  avg(c1_0.SALDO_ACTUAL)
from
  CUENTA c1_0
where
  c1_0.ES_AHORRO=0
group by
  c1_0.OFICINA_CODIGO
Oficina: 1 | Saldo Medio: 500.0 euros
Oficina: 99 | Saldo Medio: 500.0 euros
* Usando SQL Nativo:
Hibernate:
SELECT
  OFICINA_CODIGO,
  AVG(SALDO_ACTUAL)
FROM
  CUENTA
WHERE
  ES_AHORRO = 0
GROUP BY
  OFICINA_CODIGO
Oficina: 1 | Saldo Medio: 500 euros
Oficina: 99 | Saldo Medio: 500 euros

```

4. COMPROBACION DE SALDOS FINALES

```

Hibernate:
select
  c1_0.IBAN,
  c1_0.ES_AHORRO,
  c1_0.FECHA_CREACION,
  c1_0.NUMERO,
  o1_0.CODIGO,
  o1_0.DIRECCION,
  o1_0.TELEFONO,
  c1_0.SALDO_ACTUAL,
  c1_0.TIPO_INTERES
from
  CUENTA c1_0
left join
  OFICINA o1_0
  on o1_0.CODIGO=c1_0.OFICINA_CODIGO
where

```

```
c1_0.IBAN=?
Hibernate:
  select
    c1_0.IBAN,
    c1_0.ES_AHORRO,
    c1_0.FECHA_CREACION,
    c1_0.NUMERO,
    o1_0.CODIGO,
    o1_0.DIRECCION,
    o1_0.TELEFONO,
    c1_0.SALDO_ACTUAL,
    c1_0.TIPO_INTERES
  from
    CUENTA c1_0
  left join
    OFICINA o1_0
      on o1_0.CODIGO=c1_0.OFICINA_CODIGO
  where
    c1_0.IBAN=?
Cuenta corriente: ES9199001111222233334444 | Saldo: 500 euros
Cuenta ahorro: ES9199005555666677778888 | Saldo: 300 euros
abr 09, 2026 11:35:01 A. M.
org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:oracle:thin:@localhost:1521/XEPDB1]

Ejecución finalizada y conexión cerrada.
```